



Facultad de Ingeniería
Ingeniería de Sistemas e Informática y Software

Tema:

“Desarrollo de un Sistema Web Integral para mejorar la gestión administrativa en el Gimnasio STRKE”

Autor(es):

Alabrin Huaman Edson Jair U21304346

Flores Hilares Yimy Jhon U22211273

Montes Cañapataña, Héctor U22249054

Saavedra Guadalupe Yadhira Patricia U22314068

Docente:

Ing. Milla Flores, José Luis

Lima - Perú

21 de julio del 2026

Dedicatoria

A nuestras familias, por su apoyo incondicional y constante motivación durante nuestra formación académica. A todas las personas que creyeron en nuestras capacidades y nos impulsaron a alcanzar nuestras metas.

Los autores

Agradecimientos

El presente trabajo nos gustaría agradecer a nuestro docente; José Luis Milla Flores, por su orientación y enseñanza durante el curso, así como a los integrantes del equipo, por su esfuerzo, dedicación y compromiso académico.

Los autores

Resumen

El presente proyecto tuvo como objetivo desarrollar un sistema web de gestión para el gimnasio STRKE aplicando herramientas modernas de desarrollo colaborativo, control de versiones e integración continua. Para ello, se emplearon tecnologías como Git, GitHub, Git Flow, GitHub Projects, Jira, Trello y GitHub Actions, permitiendo gestionar de manera organizada el ciclo de vida del software.

Durante el desarrollo se implementaron prácticas de ingeniería de software orientadas a garantizar la trazabilidad y calidad del proyecto, incluyendo el uso de ramas feature, Pull Requests, revisión de código, gestión de issues, integración continua y análisis automatizados de calidad y seguridad. Asimismo, se incorporaron herramientas como JUnit, Mockito, JaCoCo, SonarCloud, Snyk, Trivy y Docker para fortalecer los procesos de validación, monitoreo y despliegue de la aplicación.

Como resultado, se logró construir un sistema web funcional para la administración de afiliados, membresías, asistencia y pagos del gimnasio STRKE, manteniendo un flujo de trabajo colaborativo y controlado mediante herramientas profesionales utilizadas en la industria del software.

Finalmente, el proyecto permitió aplicar de manera práctica conceptos de control de versiones, trabajo colaborativo, integración continua y DevSecOps, evidenciando la importancia de estas herramientas para mejorar la calidad, seguridad y mantenibilidad de los proyectos de software.

Palabras Clave: Git, GitHub, Git Flow, Integración Continua, DevSecOps, Docker, Desarrollo Colaborativo.

Abstract

The objective of this project was to develop a web-based management system for the STRKE gym by applying modern collaborative development tools, version control practices, and continuous integration techniques. To achieve this, technologies such as Git, GitHub, Git Flow, GitHub Projects, Jira, Trello, and GitHub Actions were used, enabling the organized management of the software development life cycle.

During the development process, software engineering practices were implemented to ensure project traceability and quality, including the use of feature branches, Pull Requests, code reviews, issue management, continuous integration, and automated quality and security analysis. In addition, tools such as JUnit, Mockito, JaCoCo, SonarCloud, Snyk, Trivy, and Docker were incorporated to strengthen application validation, monitoring, and deployment processes.

As a result, a functional web-based system was developed for managing gym members, memberships, attendance, and payments, while maintaining a collaborative and controlled workflow through professional tools widely used in the software industry.

Finally, the project provided practical experience in version control, collaborative development, continuous integration, and DevSecOps practices, highlighting the importance of these tools in improving the quality, security, and maintainability of software projects.

Keywords: web system, administrative management, gym, membership control, attendance control, process automation, decision making, information systems

TABLA DE CONTENIDO

Dedicatoria	2
Agradecimientos	3
Resumen	4
Abstract	5
Introducción	9
CAPÍTULO 1.....	10
Fundamentos del proyecto	10
1.1. Descripción del proyecto	10
1.2. Realidad Problemática	10
1.3. Justificación.....	10
1.4. Objetivos	11
1.4.1. Objetivo General	11
1.4.2. Objetivos Específicos	11
1.5. Alcances y limitaciones	12
1.5.1. Alcances.....	12
1.5.2. Limitaciones.....	12
1.6. Requerimientos del sistema	13
1.6.1. Requerimientos funcionales	13
1.6.2. Requerimientos No Funcionales.....	14
CAPÍTULO 2.....	15
Marco Teórico	15
2.1. Antecedentes	15
2.2. Bases teóricas	16
2.2.1. Sistema Web	16
2.2.2. Sistema de Gestión.....	17
2.2.3. Tecnologías utilizadas	17
2.2.3.1. Lenguaje de programación	17
2.2.3.2. Framework.....	17
2.2.3.3. Base de datos	17
2.2.3.4. Tecnologías de interfaz	17
2.2.4. Control de Versiones (Git)	18
2.2.5. Plataforma de Desarrollo Colaborativo (GitHub).....	18
a. Git Flow.....	19

b. Características Git Flow aplicadas	19
• Rama main y develop protegidas	19
• Ramas feature/* para cada tarea	20
• PR de feature a develop con Code Review.....	20
• Tag en main marcando la versión	21
• Issues cerrados con Closes #N	21
• Tablero Kanban activo	21
2.2.6. Integración Continua y DevSecOps (CI/CD)	22
2.2.6.1. Integración Continua	22
2.2.6.2. DevSecOps	22
2.2.6.3. Herramientas de CI/CD utilizadas en el proyecto	22
CAPÍTULO 3.....	24
Metodología.....	24
3.1. Metodología de desarrollo	24
3.2. Uso de GitHub en el proyecto	25
CAPÍTULO 4.....	26
Gestión del proyecto	26
4.1. Creación del repositorio.....	26
4.2. Estructura del proyecto.....	26
4.3. Estrategia de Ramas	27
4.3.1. Rama main	27
4.3.2. Rama develop	27
4.3.3. Ramas feature/*	27
4.4. Gestión de commits	28
4.5. Uso de GitHub Projects (Kanban)	28
4.6. Issues y organización.....	29
4.7. Herramienta de gestión de actividades (Jira).....	30
4.8. Integración GitHub y Jira.....	31
4.9. Resolución de Merge Conflict.....	34
4.10. Revisión y aprobación del Pull Request.....	36
4.11. Herramienta de colaboración (Trello).....	39
4.12. Release y Tag.....	41
4.13. Pipeline CI/CD y DevSecOps	44
4.13.1. Herramientas integradas en el pipeline	44
4.13.2. Diagrama del pipeline	44
4.13.3. Pruebas unitarias (JUnit + Mockito).....	46
4.13.4. JaCoCo – Medición de cobertura de pruebas	47

4.13.5. SonarCloud – Análisis de calidad de código	49
4.13.6. Snyk y Trivy - Análisis de seguridad	54
4.13.7. Resultados del escaneo de Snyk	56
4.13.8. Secrets en GitHub.....	57
4.13.9. Dockerfile multi-etapa.....	58
4.13.10. Docker Hub	59
4.13.11. Ejecución completa del pipeline (GitHub Actions)	59
4.13.12. Despliegue en Railway	60
4.14. Herramientas Complementarias.....	65
CAPÍTULO 5.....	66
Implementación y Resultados.....	66
5.1. Arquitectura Funcional del sistema	66
5.2. Base de Datos.....	66
5.3. Interfaces implementadas	67
5.3.1. Inicio de Sesión.....	67
5.3.2. Dashboard General.....	68
5.3.3. Gestión de Afiliados.....	69
5.3.4. Control de Asistencia	69
5.3.5. Asignación de Membresías	70
5.3.6. Configuración de Membresías.....	70
5.3.7. Registro de Afiliados.....	71
Conclusiones	72
Referencias Bibliográficas	73

Introducción

En la actualidad, el desarrollo de software requiere no solo conocimientos de programación, sino también el uso de herramientas que faciliten el trabajo colaborativo, el control de versiones y la automatización de procesos. Estas herramientas permiten mejorar la organización del equipo, garantizar la trazabilidad de los cambios y mantener la calidad del producto desarrollado.

Con el propósito de aplicar estos conocimientos en un entorno práctico, se desarrolló un sistema web de gestión para el gimnasio STRKE, orientado a la administración de afiliados, membresías, asistencia y pagos. Para su implementación se utilizaron tecnologías como Spring Boot, Thymeleaf, Bootstrap y MySQL.

Durante el proyecto se emplearon herramientas de desarrollo colaborativo como Git y GitHub, aplicando el modelo Git Flow para la gestión de ramas y el control de versiones. Asimismo, se utilizaron Jira y Trello para la organización de tareas, Pull Requests para la revisión de código y GitHub Actions para la automatización del pipeline de integración continua. Además, se incorporaron herramientas de calidad y seguridad como JUnit, Mockito, JaCoCo, SonarCloud, Snyk y Trivy, junto con Docker para la contenerización de la aplicación. Como parte de la estrategia de despliegue, se contempla el uso de Railway como plataforma cloud para la publicación del sistema.

Finalmente, el proyecto permitió aplicar buenas prácticas de desarrollo de software y fortalecer competencias relacionadas con el trabajo colaborativo, la integración continua y la automatización de procesos, evidenciando la importancia de estas herramientas en entornos profesionales.

CAPÍTULO 1

Fundamentos del proyecto

1.1. Descripción del proyecto

El proyecto consiste en el desarrollo de un sistema web para mejorar la gestión administrativa para el gimnasio STRKE, orientado a mejorar la administración de usuarios, membresías, pagos y asistencia. La solución busca reemplazar los procesos manuales por un sistema automatizado que permita un mejor control de la información. El sistema se desarrollará utilizando Spring Boot y una base de datos relacional, aplicando buenas prácticas de desarrollo y control de versiones mediante Github.

1.2. Realidad Problemática

Desde el inicio de sus actividades, el gimnasio STRKE ha gestionado sus procesos administrativos de forma manual, lo que ha generado diversas dificultades en el control y organización de la información. Esta forma de trabajo tradicional provoca inconsistencias en el registro de usuarios, membresías, pagos y asistencia, así como errores humanos que afectan la exactitud de los datos.

Como consecuencia, en múltiples ocasiones se presentan desorganización en la gestión de membresías, falta de control en los pagos y dificultades en el seguimiento de la asistencia de los clientes, lo que impacta negativamente en la eficiencia operativa y en la calidad del servicio brindado. Asimismo, el uso de métodos manuales demanda mayor tiempo y esfuerzo por parte del personal, dificultando la toma de decisiones oportunas.

Ante la presente situación, se hace evidente la necesidad de implementar una solución tecnológica que permita automatizar los procesos, mejorar el control de la información y optimizar la gestión del gimnasio

1.3. Justificación

El desarrollo de un sistema web de gestión integral para el gimnasio STRKE contribuye a mejorar los procesos administrativos al automatizar tareas como el registro de usuarios, la gestión de membresías, el control de pagos y el seguimiento de la asistencia. Además,

permite reducir errores manuales, agilizar el trabajo del personal y facilitar una administración más ordenada y precisa de la información.

Este proyecto es relevante porque demuestra cómo la implementación de herramientas tecnológicas puede optimizar el funcionamiento interno de un gimnasio, mejorar la organización de los datos y fortalecer la toma de decisiones. Asimismo, se emplean tecnologías como Spring Boot, base de datos MySQL, Bootstrap y Thymeleaf, las cuales permiten desarrollar una aplicación web funcional, dinámica y accesible para los usuarios. De igual manera, el uso de GitHub facilita la gestión del proyecto mediante control de versiones y trabajo colaborativo.

Por lo tanto, el presente proyecto justifica su desarrollo en la necesidad de contar con una solución tecnológica moderna y funcional que permita gestionar la información de manera confiable, centralizada y accesible, contribuyendo a mejorar la eficiencia operativa y la calidad del servicio brindado a los clientes del gimnasio STRKE.

1.4. Objetivos

1.4.1. Objetivo General

- Desarrollar un sistema web de gestión para el gimnasio STRKE, aplicando herramientas de desarrollo colaborativo como Git, GitHub y control de versiones para garantizar la trazabilidad del proyecto.

1.4.2. Objetivos Específicos

- Gestionar el código fuente del proyecto mediante Git, aplicando control de versiones y manejo de ramas por módulo
- Diseñar e implementar el sistema web usando Spring Boot, Thymeleaf, Bootstrap y MySQL como stack tecnológico.
- Utilizar GitHub como plataforma colaborativa para el seguimiento de cambios, commits y revisión de código del equipo.
- Garantizar la trazabilidad del desarrollo mediante una convención de commits que vincule cada cambio con su funcionalidad correspondiente

1.5. Alcances y limitaciones

1.5.1. Alcances

El presente proyecto comprende el desarrollo de un sistema web de gestión integral para el gimnasio STRKE, orientado a mejorar la administración de sus procesos internos mediante una solución digital. El sistema contempla las siguientes funcionalidades principales:

Módulo de autenticación: permite el inicio de sesión de usuarios mediante credenciales, asegurando el acceso al panel administrativo.

Módulo de dashboard: muestra información general del sistema, como cantidad de afiliados, ingresos y estado operativo, facilitando una visión rápida del negocio.

Módulo de afiliados: permite registrar, modificar, eliminar y consultar información de los clientes, incluyendo datos personales y estado de membresía.

Módulo de asistencia: permite registrar y consultar la asistencia de los afiliados, facilitando el control de ingreso al gimnasio.

Módulo de membresías: permite asignar, gestionar y visualizar membresías activas, incluyendo duración, precio y estado.

Configuración de membresías: permite crear, editar y eliminar tipos de membresías según las necesidades del gimnasio.

Interfaz web intuitiva: el sistema contará con una interfaz amigable desarrollada con Bootstrap y Thymeleaf, facilitando su uso por parte del personal administrativo.

1.5.2. Limitaciones

Limitación por tiempo de desarrollo

Debido al tiempo reducido del proyecto, no será posible implementar funcionalidades avanzadas, centrándose únicamente en los módulos principales como afiliados, membresías, pagos y asistencia.

Limitación tecnológica

El sistema se desarrollará como una aplicación web utilizando Spring Boot, Thymeleaf y Bootstrap, sin incluir versión móvil o aplicación nativa.

Limitación de recursos

No se integrarán tecnologías adicionales como sistemas biométricos o dispositivos automatizados para el control de acceso, debido a la disponibilidad limitada de recursos.

Limitación en infraestructura

La base de datos será gestionada en un entorno local mediante MySQL, sin implementación en servidores en la nube ni acceso remoto, lo que limita su escalabilidad.

Limitación de alcance funcional

El sistema está enfocado a la gestión de un solo gimnasio y no contempla funcionalidades avanzadas como análisis de datos, integración con sistemas externos o automatización compleja.

1.6. Requerimientos del sistema

1.6.1. Requerimientos funcionales

Los requerimientos funcionales describen las funcionalidades que el sistema debe cumplir para satisfacer las necesidades del gimnasio STRKE. Estos se derivan directamente de la problemática identificada y de los procesos del negocio.

Lista de requerimientos funcionales

- **RF01:** Iniciar sesión en el sistema
- **RF02:** Registrar afiliados
- **RF03:** Editar afiliados
- **RF04:** Eliminar afiliados
- **RF05:** Consultar afiliados
- **RF06:** Gestionar membresías

- **RF07:** Asignar membresías
- **RF08:** Registrar pagos
- **RF09:** Consultar historial de pagos
- **RF10:** Registrar asistencia
- **RF11:** Consultar asistencia
- **RF12:** Visualizar Dashboard

Estos requerimientos permiten automatizar los procesos manuales actuales, mejorando la organización de la información, reduciendo errores humanos y facilitando la gestión administrativa del gimnasio.

1.6.2. Requerimientos No Funcionales

Los requerimientos no funcionales establecen las características de calidad que debe cumplir el sistema para garantizar su correcto funcionamiento, rendimiento y usabilidad.

Lista de requerimientos no funcionales

- **RNF01:** Interfaz intuitiva y amigable (Bootstrap)
- **RNF02:** Acceso seguro mediante autenticación
- **RNF03:** Tiempo de respuesta menor a 5 segundos
- **RNF04:** Disponibilidad en horario operativo
- **RNF05:** Arquitectura en capas con Spring Boot
- **RNF06:** Escalabilidad para futuras mejoras
- **RNF07:** Compatibilidad con navegadores modernos
- **RNF08:** Integridad de datos en MySQL

CAPÍTULO 2

Marco Teórico

2.1. Antecedentes

Para la realización de este proyecto, se tomaron en consideración diversos trabajos previos relacionados con el desarrollo de sistema web de gestión, tales como tesis y proyectos de pregrado que abordan temáticas similares al objetivo del estudio. Estas investigaciones permitieron conocer diferentes enfoques y soluciones aplicadas en la administración de usuarios, control de membresías, gestión de pagos y seguimiento de asistencia en entorno como gimnasios y otros servicios.

A continuación, se mencionan estudios que han servido como referencia y que aportan antecedentes para la propuesta de Desarrollo de un Sistema Web Integral para mejorar la gestión administrativa en el gimnasio STRKE.

En el contexto internacional; en la tesis titulada “Aplicación web para mejorar la gestión administrativa del departamento de vinculación de la universidad estatal de bolívar” publicado por la Universidad Regional Autónoma de los Andes, Ambato-Ecuador, 2015 tuvo como finalidad mejorar la gestión de programas y proyectos mediante la implementación de un sistema informático que permita organizar, evaluar y monitorear la información. El sistema permitió optimizar los procesos y mejorar el acceso a la información, facilitando la toma de decisiones dentro de la institución. Pachala Guzmán, F. P. (2015). *Sistema de gestión de proyectos de vinculación* [Tesis de pregrado, Universidad Regional Autónoma de los Andes]. Repositorio Institucional UNIANDES.

En el ámbito nacional, en la tesis titulada “Implementación de un Sistema Web para mejorar la gestión de ventas, en la empresa Easy Construction Cr S.A.C” publicado por la Universidad Tecnológica del Perú, Lima-Perú, 2025 tuvo como finalidad optimizar el proceso de ventas mediante la automatización de tareas clave, como la gestión de clientes, control de inventario y registro de operaciones, a través de un sistema web. El sistema fue implementado utilizando tecnologías como Angular como framework de desarrollo, TypeScript como lenguaje de programación y MongoDB como base de datos. Uceda Gutiérrez, J. M. A. (2025). *Implementación de un sistema web para mejorar la gestión*

de ventas en la empresa Easy Construction Cr S.A.C. [Tesis de pregrado, Universidad Tecnológica del Perú]. Repositorio Institucional UTP.

2.2. Bases teóricas

El uso de la tecnología para apoyar los procesos de gestión administrativa se ha convertido en una necesidad fundamental para organizaciones de cualquier tamaño. En el caso de los gimnasios, el control de usuarios, membresías, pagos y asistencia es clave para mantener un adecuado funcionamiento del negocio.

Un manejo ineficiente de estos procesos puede generar desorganización, pérdida de información, errores en registros y dificultades en la atención al cliente. Por ello, los sistemas web se han consolidado como herramientas que permiten optimizar la gestión, mejorar la precisión de los datos y facilitar la toma de decisiones.

A partir de este contexto, el presente marco teórico expone los conceptos fundamentales que sustentan el desarrollo del sistema web de gestión integral para el gimnasio STRKE, así como las tecnologías empleadas para su implementación.

2.2.1. Sistema Web

“Un sistema web es una aplicación que se ejecuta en un navegador y permite a los usuarios acceder, procesar y gestionar información a través de internet o una red interna” (Crea System, 2022). Su propósito es facilitar el acceso a la información, optimizar procesos y mejorar la interacción entre el usuario y el sistema. En el contexto de este proyecto, el sistema web permitirá gestionar la información del gimnasio STRKE, incluyendo afiliados, membresías, pagos y control de asistencia, brindando datos organizados y accesibles para una mejor administración.

Componentes del sistema web aplicado en el proyecto:

- **Software:** aplicación web desarrollada con Spring Boot.
- **Base de datos:** MySQL para almacenar la información de forma estructurada.
- **Usuarios:** administrador y/o personal autorizado del gimnasio.
- **Procesos:** registro, actualización y consulta de afiliados, membresías, pagos y asistencia.

2.2.2. Sistema de Gestión

“Un sistema de gestión es un conjunto de herramientas que permite administrar información y procesos dentro de una organización”. Su propósito es mejorar la eficiencia y organización de las actividades. En este proyecto, el sistema de gestión permitirá organizar y controlar las operaciones del gimnasio STRKE.

2.2.3. Tecnologías utilizadas

2.2.3.1. Lenguaje de programación

Java: Java es un lenguaje orientado a objetos que permite desarrollar aplicaciones complejas y portables. Este proyecto se utiliza para construir la interfaz gráfica y la lógica del sistema, permitiendo la interacción entre el usuario y la base de datos

2.2.3.2. Framework

Spring Boot: Es un framework de código abierto que da soporte para el desarrollo de aplicaciones y páginas webs basadas en Java.

2.2.3.3. Base de datos

MySQL: MySQL Workbench es una interfaz gráfica de usuario (GUI) o herramienta de gestión gráfica que permite interactuar con la base de datos de MySQL Server, asimismo es utilizado para almacenar toda la información. Se eligió MySQL porque es una base de datos eficiente, segura, gratuita y ampliamente utilizada en entornos empresariales. Permite manejar grandes volúmenes de información sin comprometer la velocidad del sistema.

2.2.3.4. Tecnologías de interfaz

Thymeleaf: Es un moderno motor de plantillas Java del lado del servidor, apto tanto para entornos web como para entornos independientes.

Bootstrap: Bootstrap es un framework CSS de código abierto orientado a la creación de interfaces de usuario para la web.

2.2.4. Control de Versiones (Git)

“Es un sistema de control de versiones que permite a los desarrolladores registrar y gestionar los cambios realizados en el código, facilitando el trabajo colaborativo en proyectos” (The Forage, 2024). Su propósito es mantener un historial de versiones, permitiendo recuperar cambios anteriores y organizar el desarrollo de software. En el contexto de este proyecto, Git será utilizado para controlar las versiones del sistema web del gimnasio STRKE, registrando los avances realizados durante su desarrollo y permitiendo una mejor organización del código

Componentes aplicados en el proyecto:

- **Repositorio:** almacenamiento del código del sistema.
- **Commits:** registro de cambios realizados.
- **Ramas:** desarrollo de funcionalidades de manera independiente.
- **Historial:** seguimiento de versiones del proyecto.

2.2.5. Plataforma de Desarrollo Colaborativo (GitHub)

“GitHub es una plataforma de desarrollo colaborativo que aloja proyectos en la nube utilizando el sistema de control de versiones Git, permitiendo a los desarrolladores almacenar, administrar y llevar un registro de los cambios en el código” (EBAC, 2023). Su propósito es facilitar el trabajo en equipo, el seguimiento del progreso y la colaboración entre desarrolladores. En el contexto de este proyecto, GitHub será utilizado para almacenar el código fuente del sistema web del gimnasio STRKE, gestionar los cambios realizados y organizar el desarrollo mediante el uso de repositorios y ramas.

Componentes aplicados en el proyecto:

- **Repositorio:** almacenamiento del código del sistema en la nube.
- **Commits:** registro de cambios realizados en el proyecto.
- **Ramas:** desarrollo de funcionalidades de forma independiente.
- **Issues:** seguimiento de tareas y problemas.

a. Git Flow

Es un modelo de trabajo para Git, creado por Vincent Driessen en 2010, que estructura la gestión de ramas para proyectos de software. Utiliza ramas dedicadas (main, develop, feature, release, hotfix) para organizar el desarrollo, permitiendo trabajar en paralelo y gestionar lanzamientos de forma ordenada y profesional.

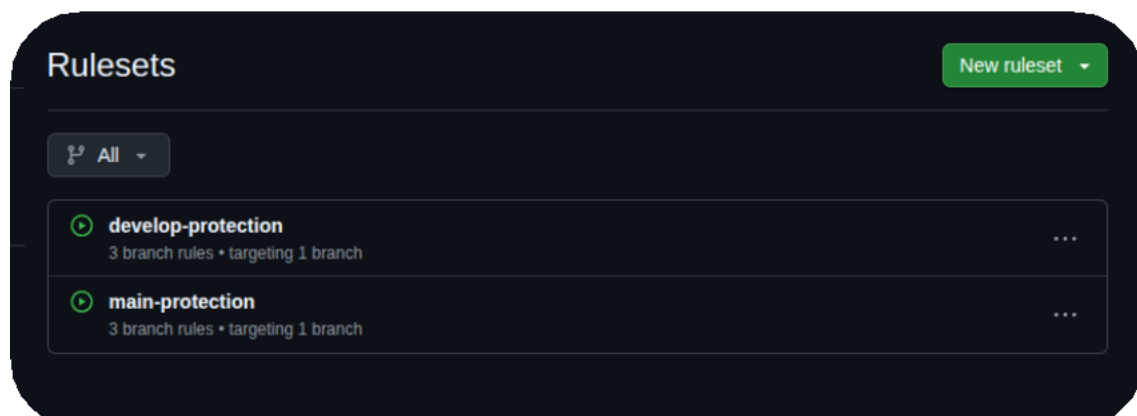


Aplicamos un Git Flow completo:

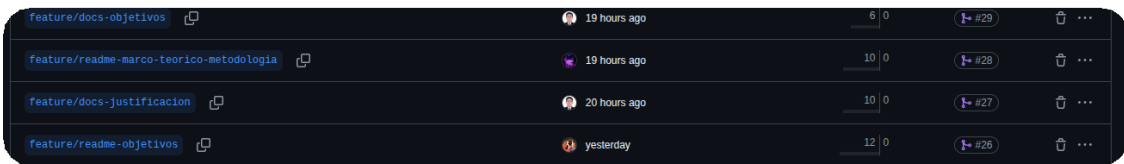
feature/* → develop → release/v1.0-entrega-parcial-1 → main
tag: v1.0-entrega-parcial-1

b. Características Git Flow aplicadas

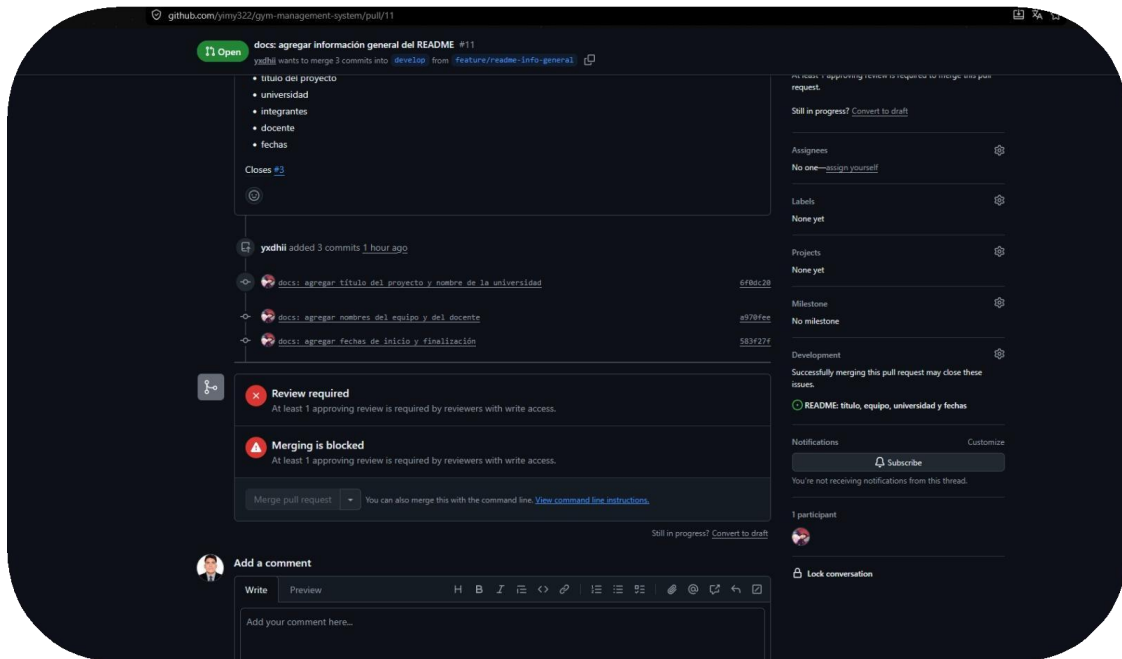
- *Rama main y develop protegidas*



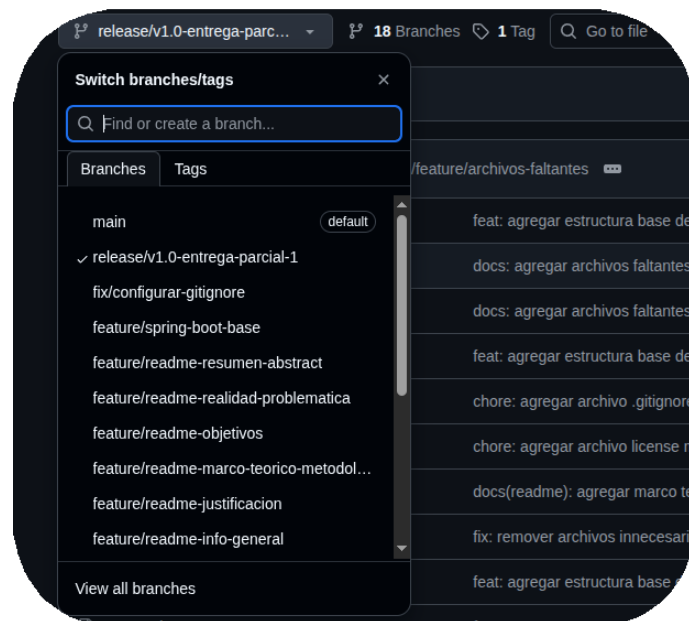
- *Ramas feature/* para cada tarea*



- *PR de feature a develop con Code Review*



- *Rama release para preparar la entrega*



- *Tag en main marcando la versión*

```
yiny@PC-322:~/universidad/ciclo-8-prt2/Herramientas-desarrollo/gym-management-system$ git tag -a v1.0-entrega-parcial-1 -m "Avance 1 completo"
yiny@PC-322:~/universidad/ciclo-8-prt2/Herramientas-desarrollo/gym-management-system$ git push origin v1.0-entrega-parcial-1
Username for 'https://github.com': yiny322
Password for 'https://yiny322@github.com':
Enumerating objects: 1, done.
Counting objects: 100% (1/1), done.
Writing objects: 100% (1/1), 171 bytes | 171.00 KiB/s, done.
Total 1 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com:yiny322/gym-management-system.git
 * [new tag]          v1.0-entrega-parcial-1 -> v1.0-entrega-parcial-1
```

- *Issues cerrados con Closes #N*

docs(readme): agregar justificación (Closes #9)

 yxdhii created `feature/readme-justificacion` • bd64e19 • 21 hours ago

- *Tablero Kanban activo*

Gym Management System

View 1 + New view

Filter by keyword or by field

Todo 0	In Progress 0	Done 16
This item hasn't been started	This is actively being worked on	This has been completed
		- gym-management-system #1 Agregar archivo .gitignore - gym-management-system #2 Agregar archivo LICENSE - gym-management-system #4 Subir cascarón de Spring Boot - gym-management-system #3 Proteger ramas main y develop - gym-management-system #18 Remover archivos innecesarios - gym-management-system #9 README: justificación - gym-management-system #10 README: objetivos generales y específicos - gym-management-system #11 README: marco teórico y metodología

2.2.6. Integración Continua y DevSecOps (CI/CD)

2.2.6.1. Integración Continua

La integración Continua (Continuous Integration CI) es una practica de desarrollo de software que automatiza la compilación, ejecución de pruebas y análisis del código fuente cada vez que un desarrollador sube cambios al repositorio. Su objetivo principal es detectar errores de forma temprana, antes de que estos lleguen al entorno de producción, reduciendo así el costo y el tiempo de corrección de fallos. Al integrar los cambios de manera frecuente y automatizada, el equipo de desarrollo puede identificar conflictos de código, errores de compilación y fallos en las pruebas casi de inmediato, en lugar de descubrirlos semanas después durante una integración manual.

2.2.6.2. DevSecOps

DevSecOps es la evolución de las prácticas DevOps que incorpora la seguridad como una responsabilidad compartida e integrada dentro de todo el ciclo de vida del desarrollo de software, en lugar de tratarla como una fase final y aislada. Bajo este enfoque, los análisis de seguridad (vulnerabilidades en dependencias, vulnerabilidades en imágenes de contenedores, malas prácticas de código, entre otros) se ejecutan automáticamente en cada push o pull request, junto con las pruebas funcionales. Esto permite detectar y corregir riesgos de seguridad en etapas tempranas del desarrollo, evitando que código vulnerable llegue a producción.

2.2.6.3. Herramientas de CI/CD utilizadas en el proyecto

Para implementar el pipeline de CI/CD y DevSecOps del proyecto se integraron las siguientes herramientas, cada una cumpliendo un rol específico dentro del flujo de automatización:

Herramientas	Función
GitHub Actions	Motor de orquestación que ejecuta automáticamente el pipeline ante cada push o Pull Request.
Junit + Mockito	Framework de pruebas unitarias para validar la lógica de negocio del backend
JaCoCo	Plugin de Maven que mide el porcentaje de cobertura de código ejecutado por las pruebas.
SonarCloud	Plataforma de análisis estático de código que evalúa calidad, bugs, vulnerabilidades y duplicación.

Snyk	Herramienta de seguridad que detecta vulnerabilidades conocidas en las dependencias del proyecto.
Trivy	Escáner de vulnerabilidades a nivel de sistema operativo dentro de la imagen Docker.
Docker	Tecnología de contenedores utilizada para empaquetar la aplicación junto con sus dependencias.
Railway	Plataforma de despliegue en la nube utilizada para publicar la aplicación contenerizada.

CAPÍTULO 3

Metodología

3.1. Metodología de desarrollo

El desarrollo del sistema se realizó bajo un enfoque **incremental**, dividiendo la construcción del software en módulos funcionales independientes: afiliados (miembros), membresías, pagos/suscripciones y asistencia. Cada módulo se desarrolló, probó e integró de forma progresiva al sistema, permitiendo validar funcionalidades de manera temprana sin necesidad de esperar a que el sistema completo estuviera terminado.

Este enfoque incremental se complementó con un flujo de trabajo de tipo **Kanban**, materializado mediante un tablero de GitHub Projects (*ver sección 4.5*) y reforzado con Jira y Trello como herramientas de apoyo en la organización de tareas (*ver secciones 4.7 y 4.11*). Cada módulo se descompuso en tareas o *issues* concretos, los cuales se movieron a través de columnas de estado (*To Do, In Progress, Done*) a medida que avanzaba el desarrollo, lo que permitió tener visibilidad constante del progreso del proyecto y priorizar el trabajo según las dependencias entre módulos.

La combinación de desarrollo incremental con un tablero Kanban ofreció las siguientes ventajas al proyecto:

- **Entregas progresivas:** cada módulo (afiliados, membresías, pagos, asistencia) pudo probarse de forma aislada antes de integrarse con el resto del sistema.
- **Detección temprana de errores:** al validar cada incremento por separado, los defectos se identificaron y corrigieron antes de que se propagaran a otros módulos.
- **Flexibilidad ante cambios:** al no depender de sprints de duración fija, el equipo pudo reordenar prioridades según la complejidad real de cada módulo durante el desarrollo.
- **Trazabilidad del trabajo:** cada tarea del tablero quedó vinculada a un *issue*, una rama y, posteriormente, a un *pull request*, generando un historial completo del avance del proyecto (*este flujo se detalla en el Capítulo 4*).

3.2. Uso de GitHub en el proyecto

GitHub se utilizó como la herramienta central de control de versiones y gestión colaborativa del proyecto, integrando en una sola plataforma el código fuente, la organización de tareas y la automatización del flujo de trabajo. Sobre la base del modelo Git Flow descrito en el marco teórico (sección 2.2.5), GitHub permitió:

- **Control de versiones del código fuente**, mediante repositorios, ramas (main, develop, feature/*) y commits, registrando de forma ordenada cada cambio realizado durante el desarrollo de los módulos del sistema.
- **Organización del trabajo en equipo**, mediante *Issues* vinculados a tareas específicas y un tablero Kanban (GitHub Projects), que reflejó visualmente el avance del enfoque incremental descrito en la sección 3.1.
- **Control de calidad del código**, mediante *Pull Requests* con revisión de código (*Code Review*) antes de fusionar cualquier cambio a la rama develop.
- **Automatización del pipeline de integración y despliegue**, mediante GitHub Actions, ejecutando de forma automática pruebas unitarias, análisis de calidad y seguridad, y despliegue de la aplicación ante cada cambio relevante en el repositorio.

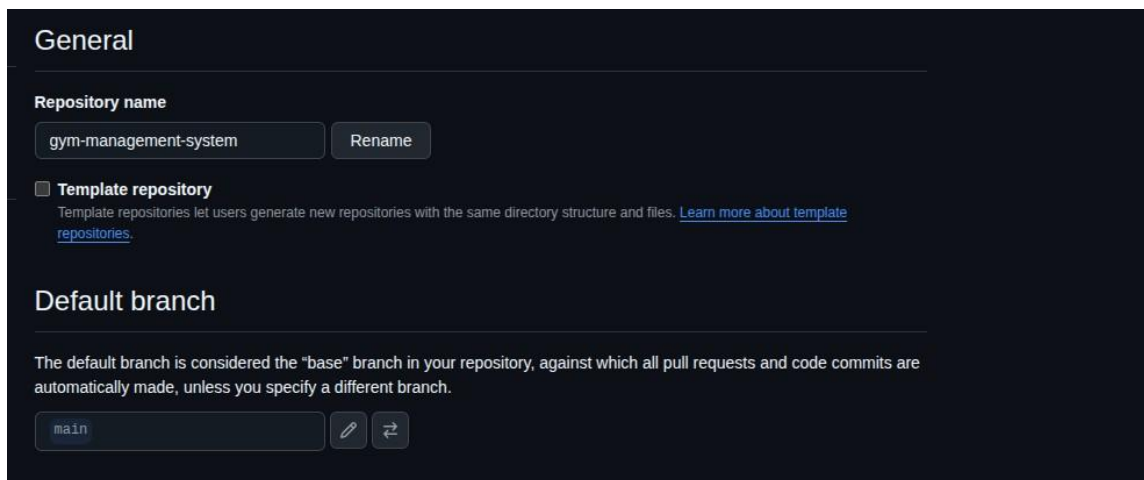
El detalle operativo de cada uno de estos usos —creación del repositorio, estrategia de ramas, gestión de commits, tablero Kanban, integración con Jira y Trello, resolución de conflictos, revisión de Pull Requests, releases y el pipeline CI/CD— se desarrolla a profundidad en el **Capítulo 4: Gestión del proyecto**.

CAPÍTULO 4

Gestión del proyecto

4.1. Creación del repositorio

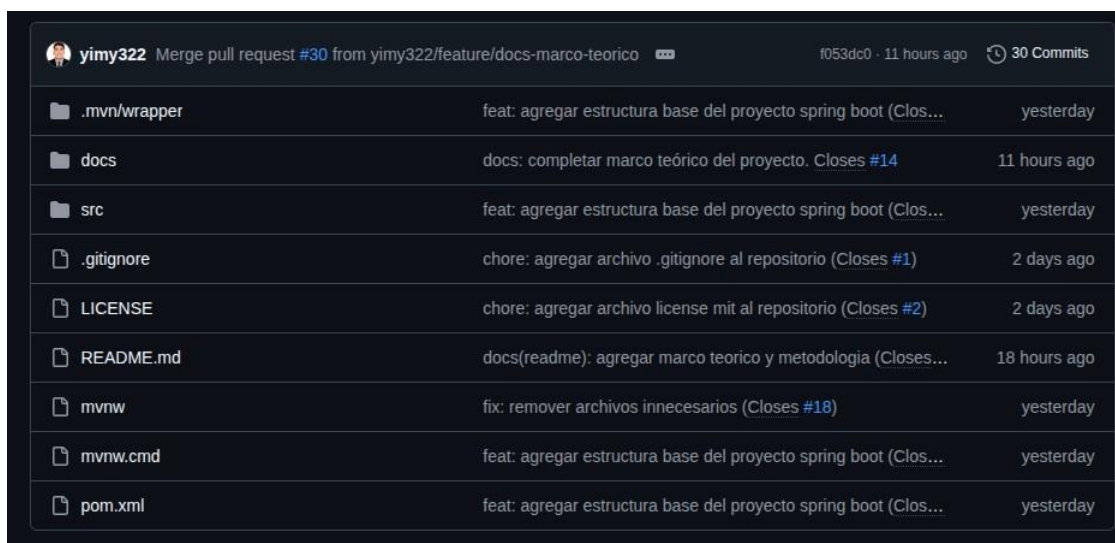
Se creó un repositorio en GitHub para almacenar el código fuente y la documentación del sistema web de gestión integral para el gimnasio STRKE. Este repositorio permitirá centralizar el desarrollo del proyecto, registrar los avances y mantener un historial de cambios.



The screenshot shows the 'General' tab of a new GitHub repository. The repository name is 'gym-management-system'. There is a 'Rename' button. The 'Template repository' checkbox is unchecked. Below it, a note says 'Template repositories let users generate new repositories with the same directory structure and files. [Learn more about template repositories.](#)'. The 'Default branch' section explains that the default branch is the 'base' branch for pull requests and commits. The default branch is set to 'main'.

4.2. Estructura del proyecto

El repositorio se organizó en carpetas para separar el código fuente, la documentación y los informes del proyecto. Esta estructura permite mantener un orden adecuado y facilitar la ubicación de los archivos.



 yimy322	Merge pull request #30 from yimy322/feature/docs-marco-teorico	f053dc0 · 11 hours ago	30 Commits
📁 .mvn/wrapper	feat: agregar estructura base del proyecto spring boot (Closes #13)	yesterday	
📁 docs	docs: completar marco teórico del proyecto. Closes #14	11 hours ago	
📁 src	feat: agregar estructura base del proyecto spring boot (Closes #13)	yesterday	
📄 .gitignore	chore: agregar archivo .gitignore al repositorio (Closes #1)	2 days ago	
📄 LICENSE	chore: agregar archivo license mit al repositorio (Closes #2)	2 days ago	
📄 README.md	docs(readme): agregar marco teorico y metodologia (Closes #3)	18 hours ago	
📄 mvnw	fix: remover archivos innecesarios (Closes #18)	yesterday	
📄 mvnw.cmd	feat: agregar estructura base del proyecto spring boot (Closes #13)	yesterday	
📄 pom.xml	feat: agregar estructura base del proyecto spring boot (Closes #13)	yesterday	

4.3. Estrategia de Ramas

Para el desarrollo del proyecto se utilizará una estrategia de ramas que permita organizar el trabajo y evitar modificaciones directas en la rama principal.

4.3.1. Rama main

La rama **main** será utilizada como rama principal del proyecto, donde se mantendrá la versión estable del sistema.

4.3.2 Rama develop

La rama **develop** será utilizada para integrar los avances del desarrollo antes de ser enviados a la rama principal.

4.3.3 Ramas feature/*

Las **ramas feature/*** se utilizarán para desarrollar funcionalidades específicas del sistema, como afiliados, membresías, pagos o asistencia.

Default					
Branch	Updated	Check status	Behind	Ahead	Pull request
main  	 4 days ago			Default	 ...
Your branches					
Branch	Updated	Check status	Behind	Ahead	Pull request
feature/archivos-faltantes 	 9 hours ago		0	29	 ...
develop  	 11 hours ago		0	29	 ...
feature/spring-boot-base 	 yesterday		0	6	 #17  ...
feature/add-gitignore 	 2 days ago		0	3	 #16  ...
feature/add-license 	 2 days ago		0	1	 ...
Active branches					
Branch	Updated	Check status	Behind	Ahead	Pull request
feature/archivos-faltantes 	 9 hours ago		0	29	 ...
develop  	 11 hours ago		0	29	 ...
feature/docs-marco-teorico 	 11 hours ago		0	28	 #30  ...
feature/docs-objetivos 	 17 hours ago		0	26	 #29  ...
feature/readme-marco-teorico-metodologia 	 18 hours ago		0	22	 #28  ...
View more branches >					

4.4. Gestión de commits

Los commits se realizarán de manera periódica para registrar los avances del proyecto. Cada commit tendrá un mensaje descriptivo que permita identificar el cambio realizado.

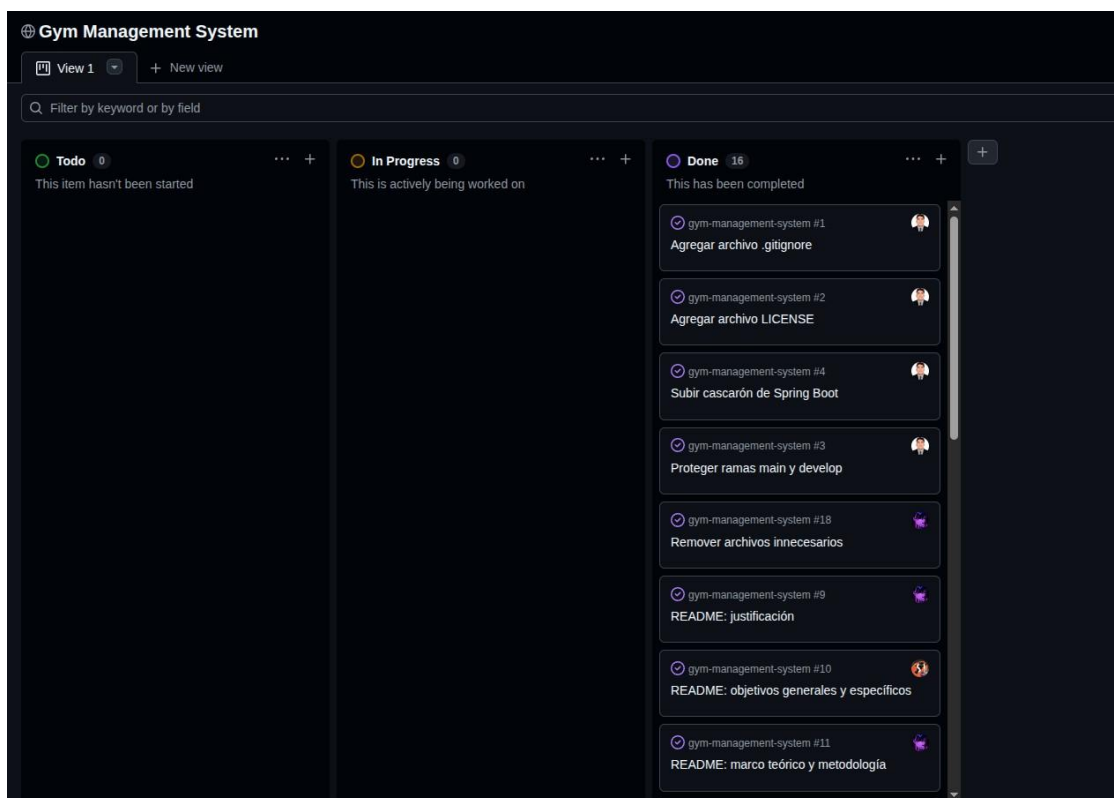
Ejemplos:

4.5 Uso de GitHub Projects (Kanban)

Se utilizará GitHub Projects como tablero Kanban para organizar las actividades del proyecto. Las tareas se clasificarán según su estado: pendientes, en proceso y finalizadas.

Estados del tablero:

- Pendiente
- En proceso
- Finalizado

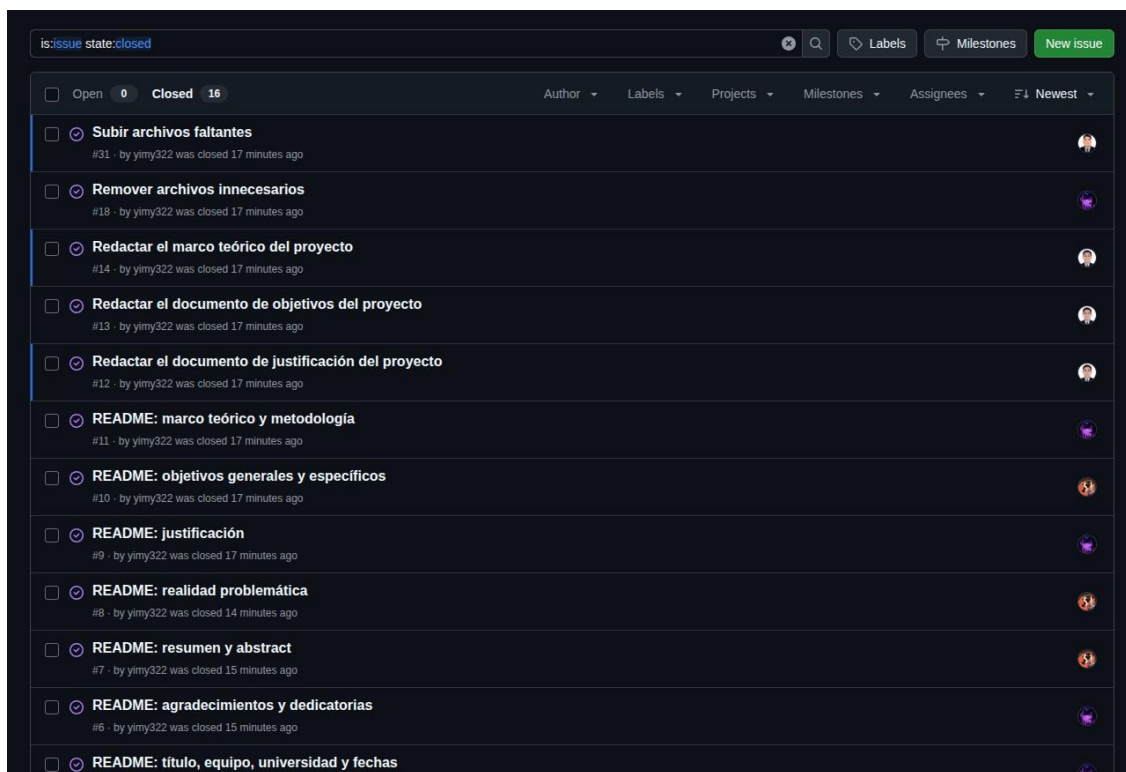


4.6. Issues y organización

Los Issues se utilizarán para registrar tareas, mejoras o problemas relacionados con el desarrollo del sistema. Cada issue permitirá dar seguimiento a una actividad específica y podrá vincularse con commits o ramas del proyecto.

Ejemplos de Issues:

- Proteger ramas main y develop
- Agregar archivo LICENSE
- README: marco teórico y metodología
- Elaborar documentación inicial
- Actualizar README.md

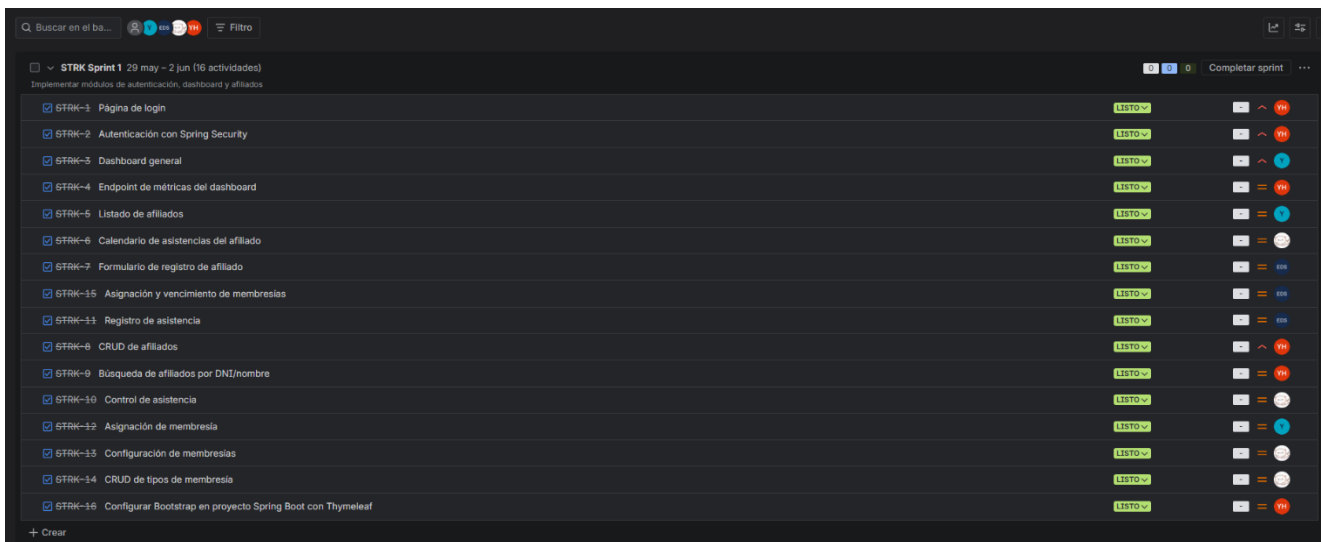


4.7. Herramienta de gestión de actividades (Jira)

Jira es una plataforma de gestión de proyectos desarrollada por Atlassian que permite planificar, rastrear y organizar el trabajo del equipo mediante tickets, sprints y tableros visuales.

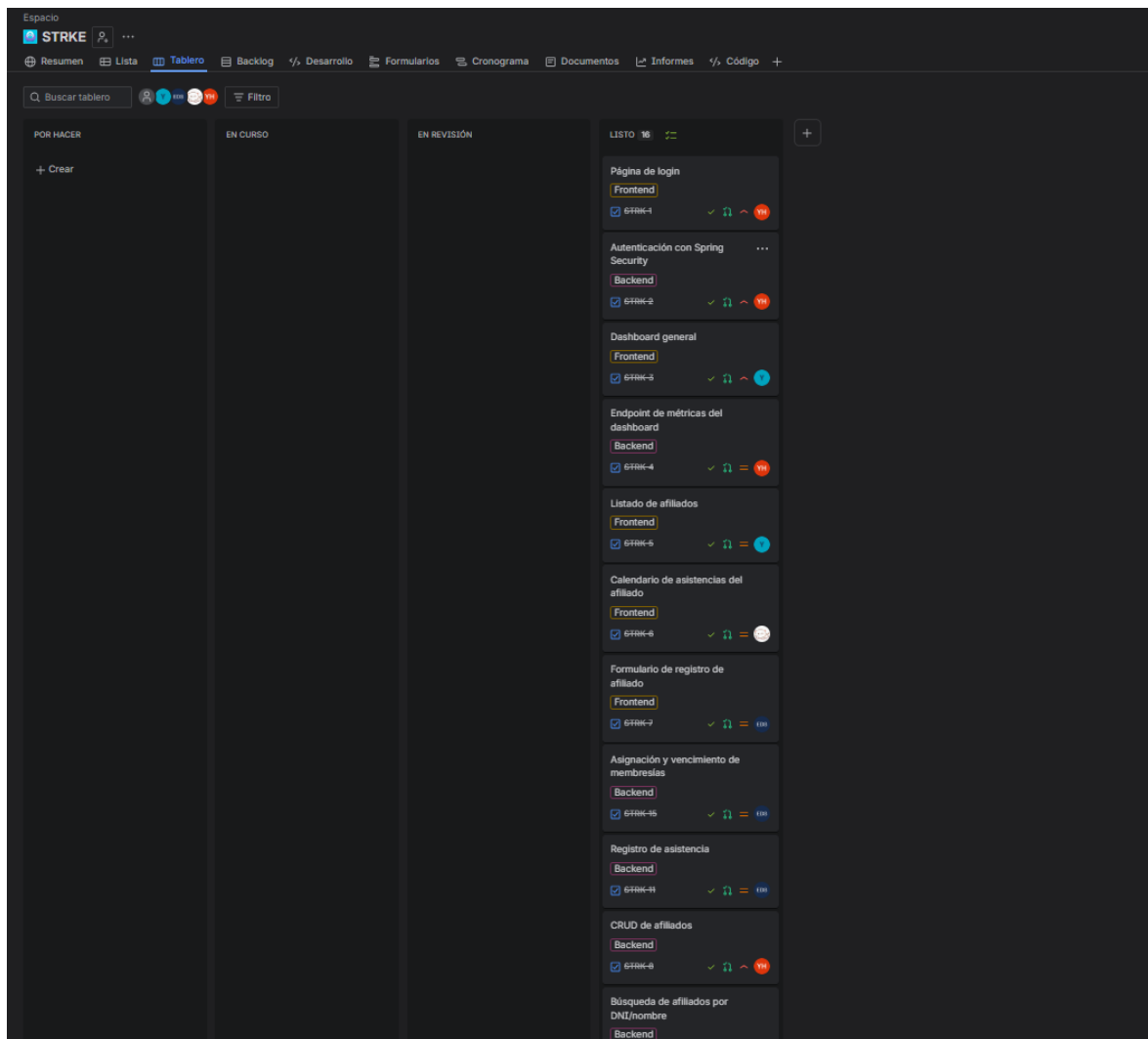
Sprint 1

Se creó el **STRK Sprint 1** con un total de 16 actividades orientadas a implementar los módulos de autenticación, Dashboard y afiliados. Al finalizar el sprint, las 16 tareas se encuentran en estado **Listo**.



Tablero Kanban

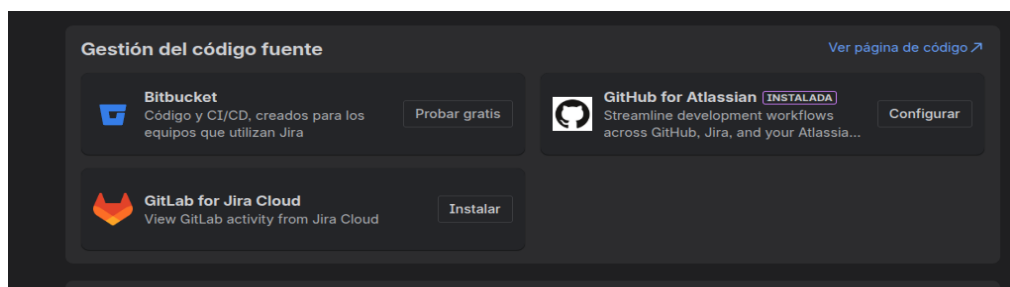
El tablero organiza las tareas en cuatro columnas según su estado: **Por hacer**, **En curso**, **En revisión** y **Listo**. Permite visualizar el avance del sprint en tiempo real identificando qué tareas están pendientes y cuáles fueron completadas.



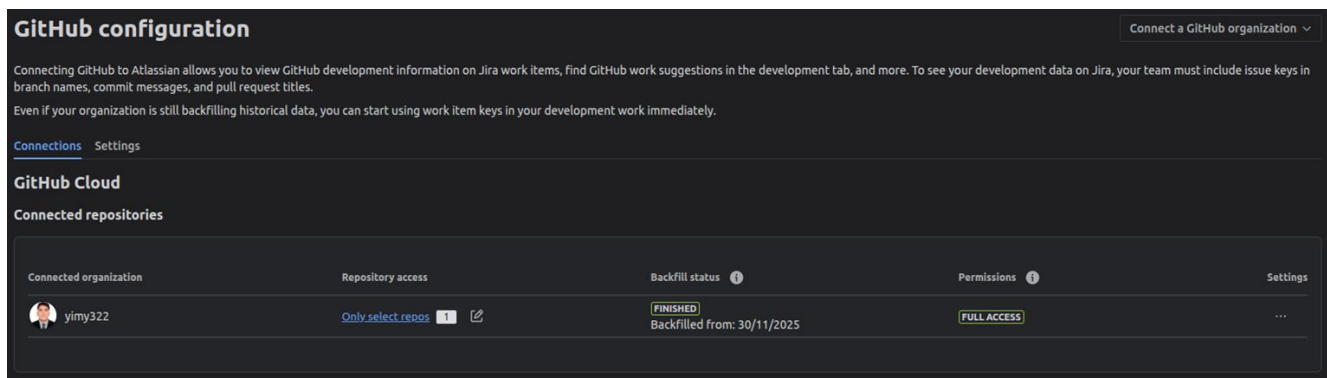
4.8. Integración GitHub y Jira

Para vincular el seguimiento de tareas con el desarrollo del código, se integró Jira con GitHub mediante el plugin **GitHub for Atlassian**.

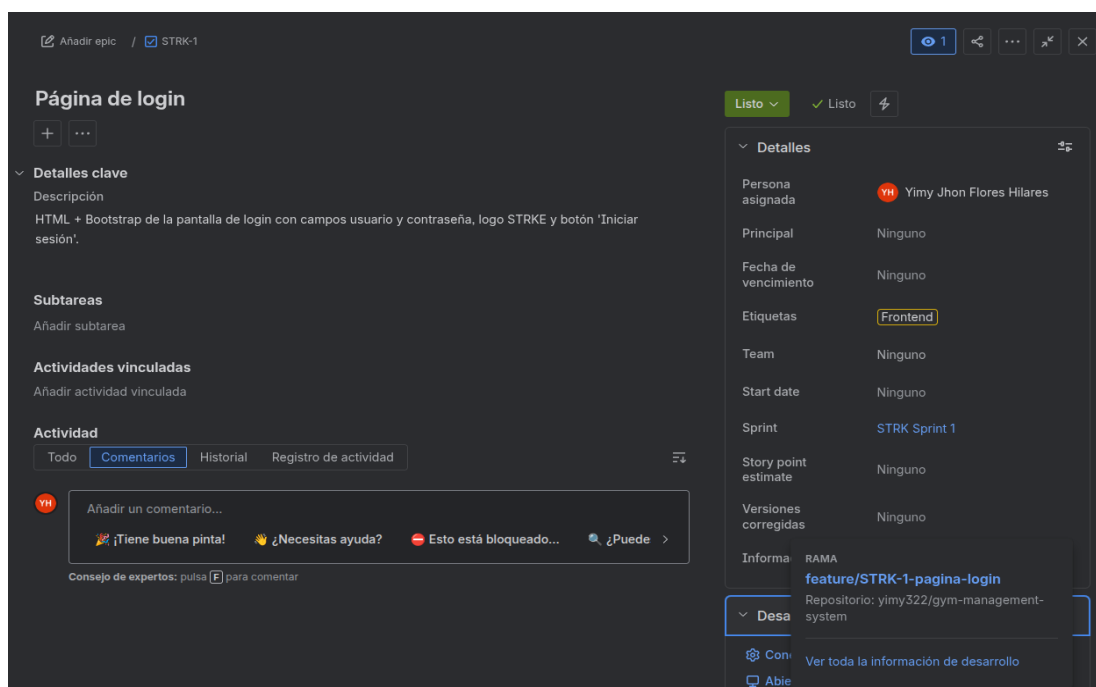
Paso 1. Instalación del plugin de GitHub for Atlassian en Jira.



Paso 2. Conectar la cuenta de GitHub y seleccionar los repositorios a vincular.



Paso 3. Usar el identificador del issue (ej. STRK-1) en el nombre de las ramas y commits para vincularlos automáticamente con su tarea en Jira.



Rama: *feature/STRK-1-pagina-login* vista desde el github



Rama: *feature/STRK-1-pagina-login* ya vinculada en el issue correspondiente gracias al identificador **STRK-1**

Desarrollo de STRK-1

Dar feedback
X

Rama
Confirmacion
Solicitudes de incorporación de ca
Compilacion
Implementacion
Indicadores de funcionalid
Otros enlaci

yimy322/gym-management-system (GitHub)

Rama	Solicitud de incorporación de cambios	Acción
feature/STRK-1-pagina-login	1 Solicitudes de incorporación de cambios 1 FUSIONADA	Crear solicitud de incorporación de cambios

Commit: feat: maquetar pagina de login con bootstrap - STRK-1 #done

Activity

feature/STRK-1-...
All activity
All users
All time

feat: maquetar pagina de login con bootstrap - STRK-1 #done
yimy322 pushed 1 commit • 4a70174...ace59ca • yesterday

Paso 4. Configuramos una regla de automatización en Jira para cerrar el issue automáticamente cuando un commit incluye #done en su mensaje.

Cerrar issue con #done

Commit creado
El flujo se ejecuta al crear una confirmación

Se ha aplicado la condición
{{commit.message}} contiene #done

Realizar la transición de la actividad a LISTO

Detalles del flujo
Los campos obligatorios están marcados con un asterisco *

Nombre * (requerido)
Cerrar issue con #done

Descripción
Añadir una descripción a tu flujo

Alcance
STRKE
Solo se puede modificar el alcance en la [administración global](#).

Propietario * (requerido)
Yimy Jhon Flores Hilaes

Agente * (requerido)
Automation for Jira

Notificar al producirse un error
Enviar un mensaje de correo electrónico al propietario del flujo una...

¿Quién puede editar este flujo? * (requerido)
Privado

☐ Marca esto para permitir que las acciones de flujo desencadenen

**** Cada issue muestra su rama y repositorio vinculado directamente desde Jira.**


```
<form th:action="@{/logout}" method="post">
  <input type="hidden" th:name="${_csrf.parameterName}" th:value="${_csrf.token}"/>
  <button type="submit" class="btn btn-sm btn-outline-secondary rounded-circle p-1" style="width:32px; height:32px;">
    <i class="fa-solid fa-arrow-right-from-bracket" style="font-size:12px;"></i>
  </button>
</form>
</div>
```

Paso 4. Commit y push

Se agregó el archivo corregido, se realizó un commit indicando la resolución del conflicto y se subió el cambio a la rama remota.

```
Yadhira Saavedra@LAPTOP-22GAESQ MINGW64 ~/Documents/STRK
$ git add .
$ git commit -m "fix: resolucio conflictto navbar logout"
$ git push origin feature/STRK-2-spring-security
```

```
yiny@PC-322:~/universidad/ciclo-8-prt2/herramientas-desarrollo/gym-management-system$ git add .
yiny@PC-322:~/universidad/ciclo-8-prt2/herramientas-desarrollo/gym-management-system$ git commit -m "fix: resolucio conflictto navbar logout"
[feature/STRK-2-spring-security 065c484] fix: resolucio conflictto navbar logout
yiny@PC-322:~/universidad/ciclo-8-prt2/herramientas-desarrollo/gym-management-system$ git push origin feature/STRK-2-spring-security
Username for 'https://github.com': yiny322
Password for 'https://yiny322@github.com':
Enumerating objects: 1, done.
Counting objects: 100% (1/1), done.
Writing objects: 100% (1/1), 227 bytes | 227.00 KiB/s, done.
Total 1 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/yiny322/gym-management-system.git
 9a6a2d8..065c484 feature/STRK-2-spring-security -> feature/STRK-2-spring-security
yiny@PC-322:~/universidad/ciclo-8-prt2/herramientas-desarrollo/gym-management-system$
```

Paso 5. Pull Request sin conflictos

Con el conflicto resuelto, se creó el Pull Request de feature/STRK-2-spring-security hacia develop, quedando a la espera de aprobación para su fusión.

Feature/strk 2 spring security #53 Awaiting approval Code

Open yiny322 wants to merge 2 commits into `develop` from `feature/STRK-2-spring-security`

Conversation 0 Commits 2 Checks 0 Files changed 8 +138 -7

yiny322 commented now Owner

No description provided.

yiny322 added 2 commits 3 hours ago

- feat: configurar spring security y manejo de sesion - STRK-2 #done 9a6a2d8
- fix: resolucio conflictto navbar logout 065c484

Review required
At least 1 approving review is required by reviewers with write access.

Merging is blocked
At least 1 approving review is required by reviewers with write access.

Merge pull request You can also merge this with the command line. [View command line instructions.](#)

Reviewers

Suggestions

yxdhil [Request](#)

At least 1 approving review is required to merge this pull request.

Still in progress? [Convert to draft](#)

Assignees

No one—[assign yourself](#)

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

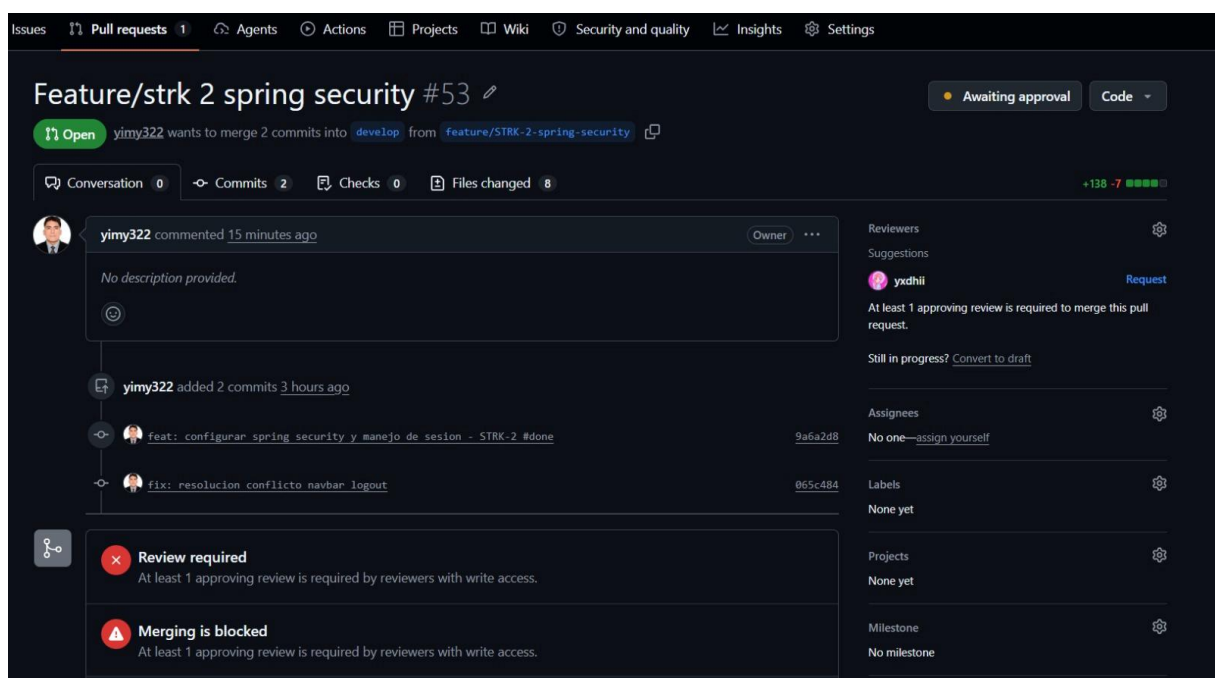
Successfully merging this pull request may close these issues.

4.10. Revisión y aprobación del Pull Request

Para garantizar la calidad del código, se estableció un flujo de revisión donde cada Pull Request requiere la aprobación de al menos un integrante del equipo antes de fusionarse.

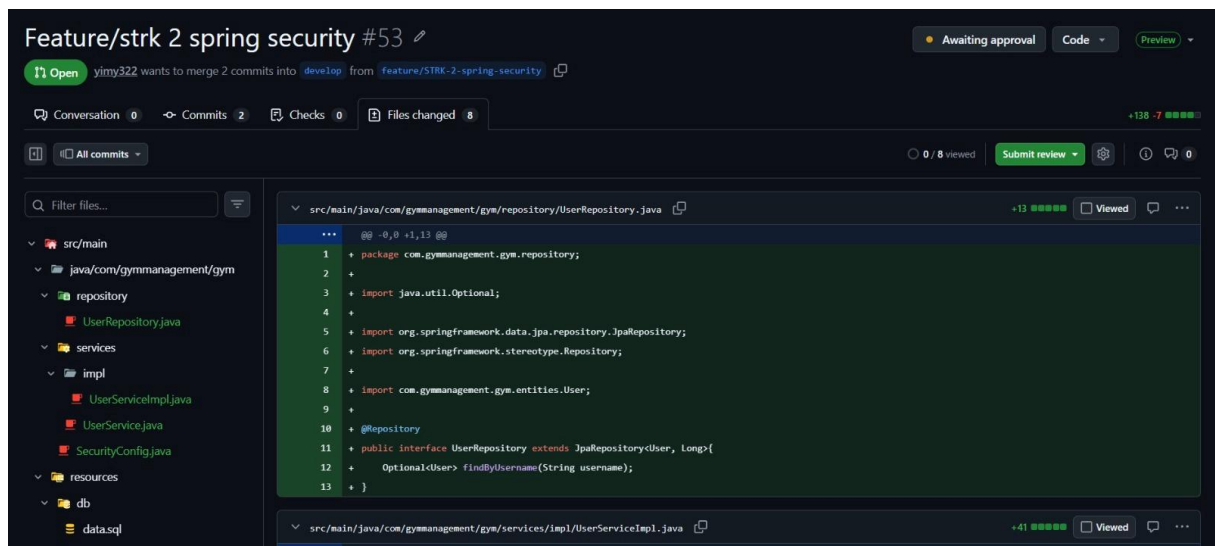
Paso 1. Apertura del Pull Request

Se abre el PR de feature/STRK-2-spring-security hacia develop con 2 commits, quedando en espera de aprobación del revisor.



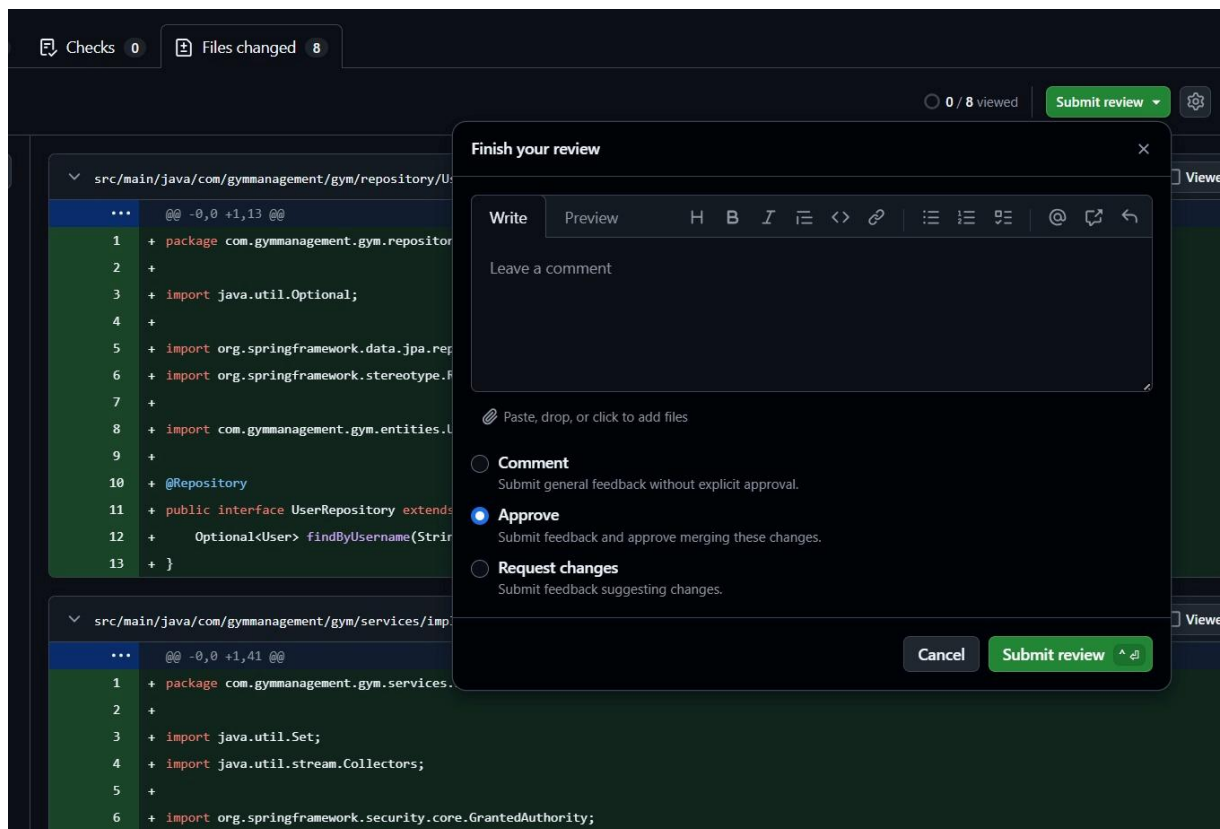
Paso 2. Revisión de archivos

El revisor accede a la pestaña Files changed para revisar los 8 archivos modificados.



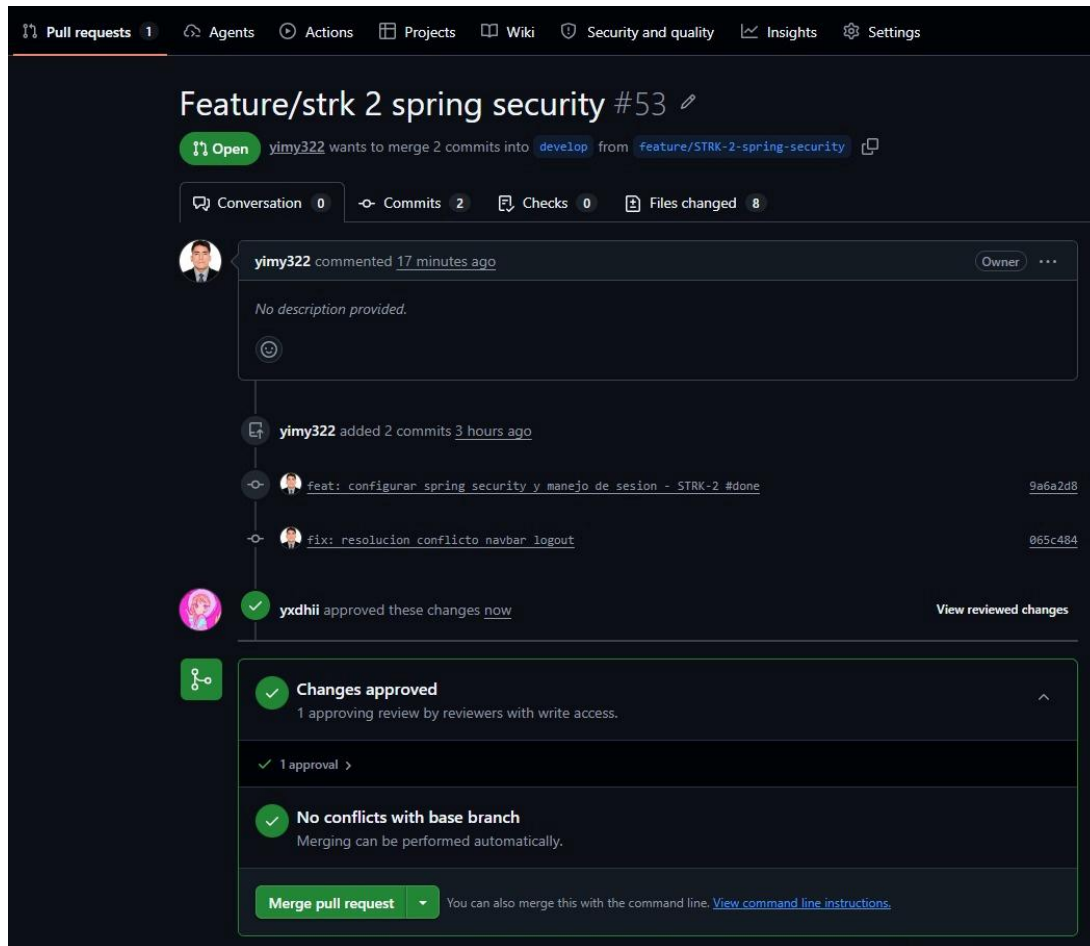
Paso 3. Aprobación

Tras revisar los archivos, el revisor selecciona Approve y hace clic en Submit review para aprobar el PR.



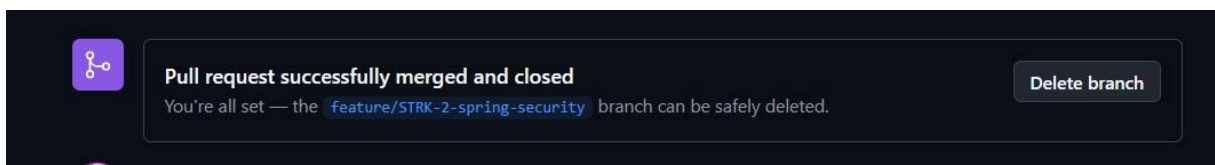
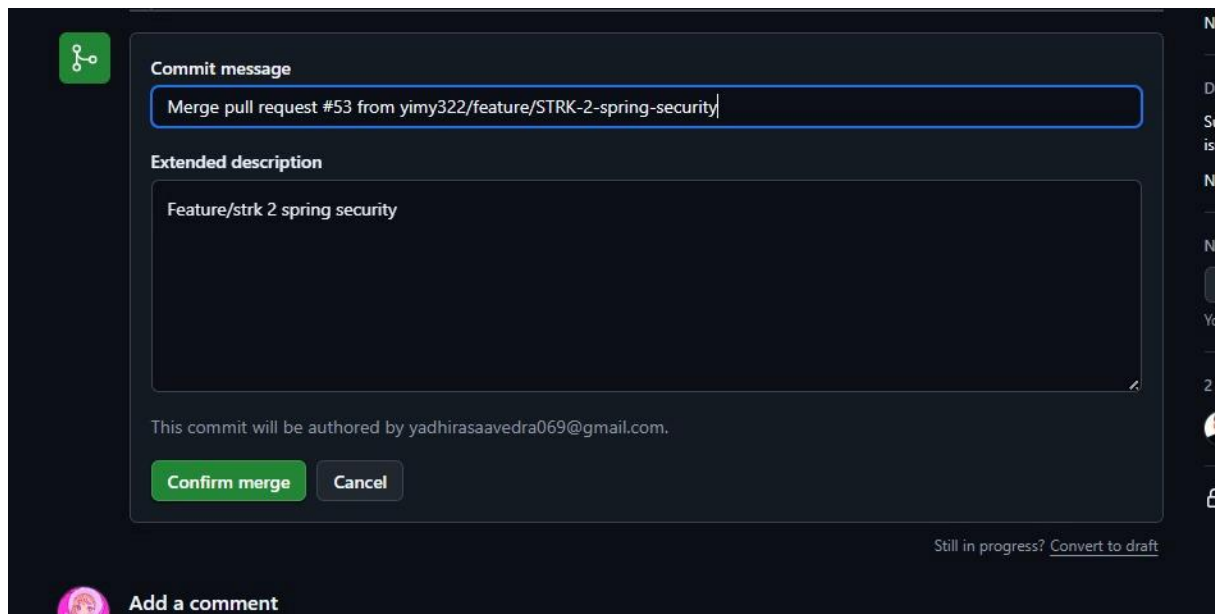
Paso 4. Merge

Una vez aprobado, el PR muestra Changes approved y No conflicts with base branch, habilitando el botón para realizar el merge.



Paso 5. Confirmación

Se confirma el merge con Confirm merge, fusionando la rama en develop exitosamente y cerrando el PR.



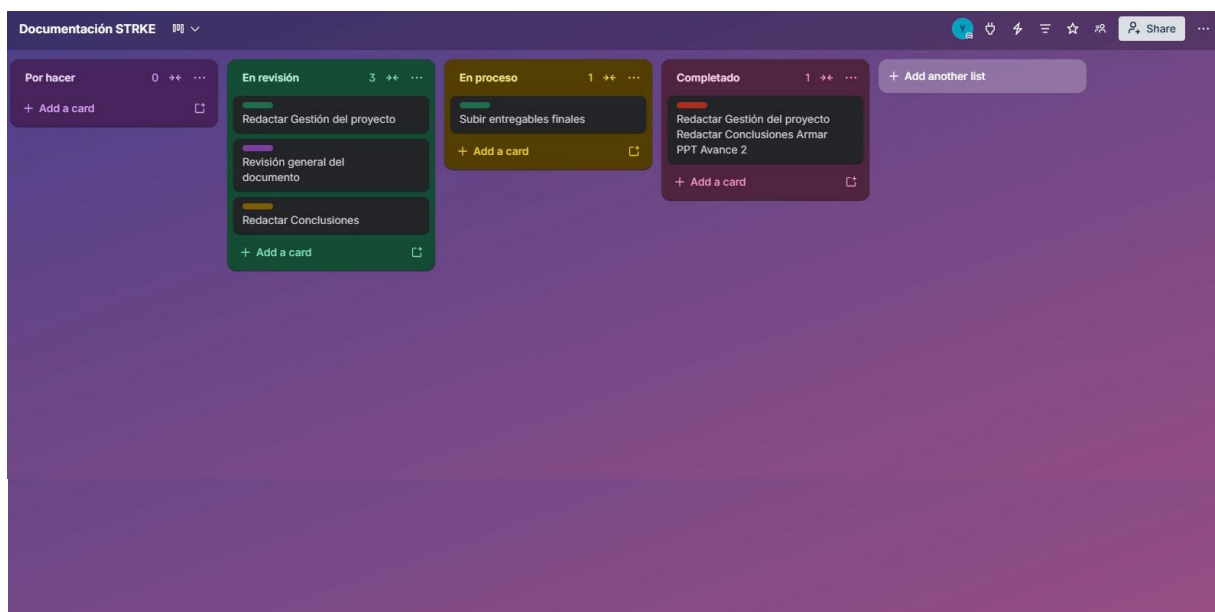
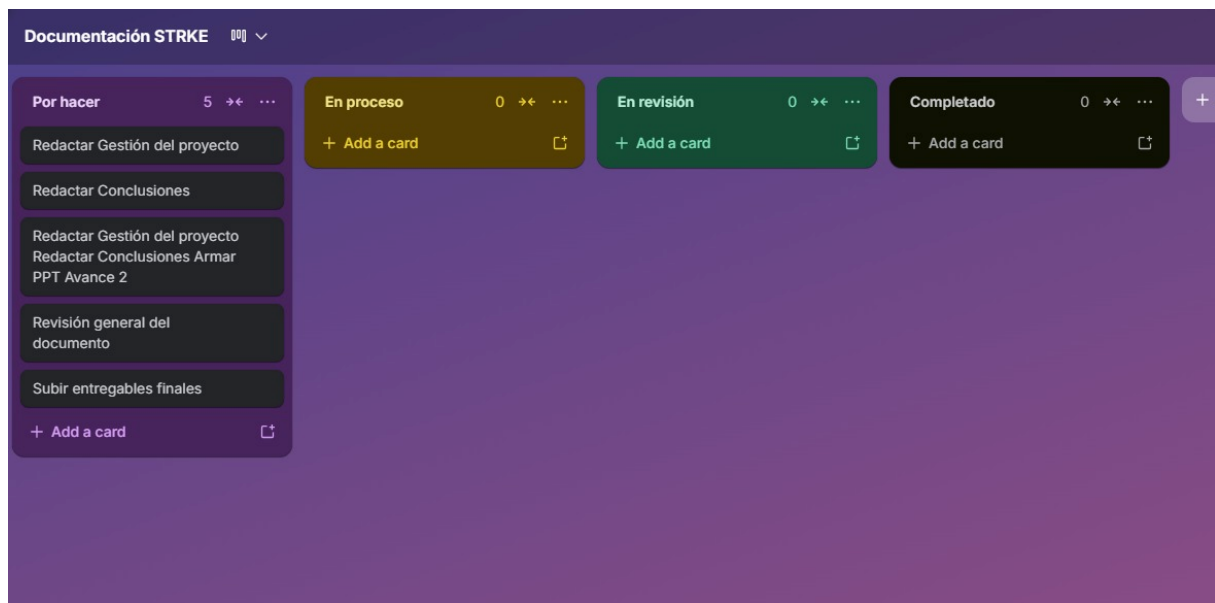
4.11. Herramienta de colaboración (Trello)

Trello es una herramienta de gestión visual de tareas basada en tableros Kanban que permite organizar actividades mediante tarjetas distribuidas en columnas según su estado. A diferencia de Jira, utilizado para el seguimiento del desarrollo del sistema, Trello se empleó para gestionar las actividades de documentación del proyecto.

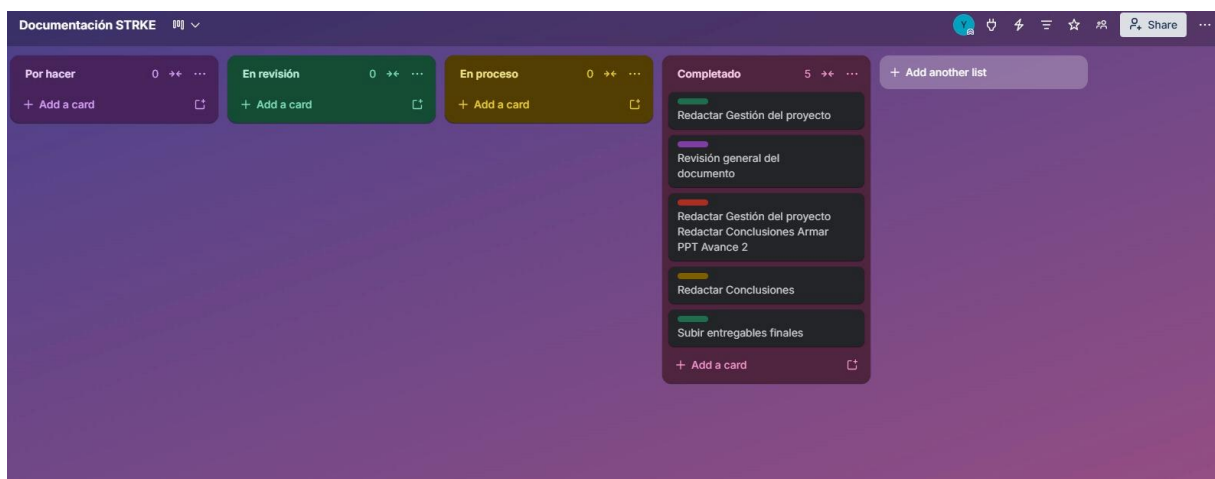
El tablero **Documentación STRKE** se organizó en cuatro columnas: **Por hacer, En proceso, En revisión y Completado**, con las siguientes actividades:

- ❖ Redactar Gestión del proyecto
- ❖ Redactar Conclusiones
- ❖ Redactar Gestión del proyecto, Redactar Conclusiones y Armar PPT Avance 2
- ❖ Revisión general del documento
- ❖ Subir entregables finales

Al cierre del avance, las 5 actividades se encuentran en estado **Completado**.



- *Actividades Completadas*



4.12. Release y Tag

Una vez completado el desarrollo del Sprint 1, se procedió a crear el release oficial del proyecto siguiendo los pasos del flujo GitFlow.

Paso 1. Nos ubicamos en la rama develop ejecutando `git checkout develop`.

```
yimy@PC-322:~/universidad/ciclo-8-prt2/herramientas-desarrollo/gym-management-system$ git checkout develop
Already on 'develop'
Your branch is up to date with 'origin/develop'.
yimy@PC-322:~/universidad/ciclo-8-prt2/herramientas-desarrollo/gym-management-system$
```

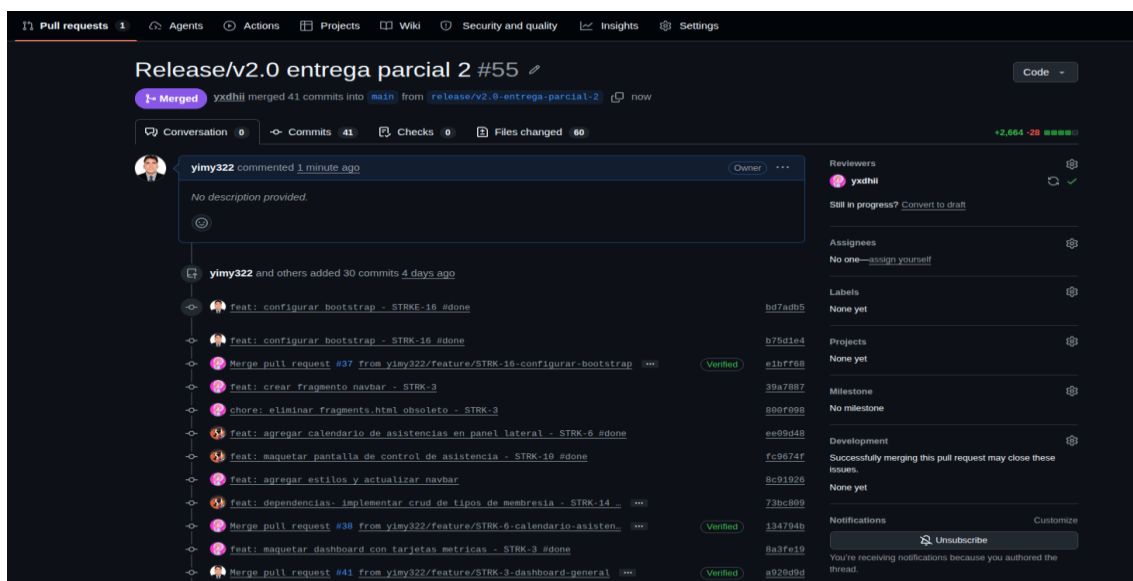
Paso 2. Se crea la rama del release ejecutando `git checkout -b release/v2.0-entrega-parcial-2`.

```
yimy@PC-322:~/universidad/ciclo-8-prt2/herramientas-desarrollo/gym-management-system$ git checkout -b release/v2.0-entrega-parcial-2
Switched to a new branch 'release/v2.0-entrega-parcial-2'
```

Paso 3. Se sube la rama al repositorio remoto ejecutando `git push -u origin release/v2.0-entrega-parcial-2`.

```
yimy@PC-322:~/universidad/ciclo-8-prt2/herramientas-desarrollo/gym-management-system$ git push -u origin release/v2.0-entrega-parcial-2
Username for 'https://github.com': yimy322
Password for 'https://yimy322@github.com':
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0
remote:
remote: Create a pull request for 'release/v2.0-entrega-parcial-2' on GitHub by visiting:
remote:   https://github.com/yimy322/gym-management-system/pull/new/release/v2.0-entrega-parcial-2
remote:
To https://github.com:yimy322/gym-management-system.git
 * [new branch]      release/v2.0-entrega-parcial-2 -> release/v2.0-entrega-parcial-2
Branch 'release/v2.0-entrega-parcial-2' set up to track remote branch 'release/v2.0-entrega-parcial-2' from 'origin'.
```

Paso 4. Se crea el Pull Request de `release/v2.0-entrega-parcial-2` hacia `main`, el revisor aprueba y se realiza el merge exitosamente.



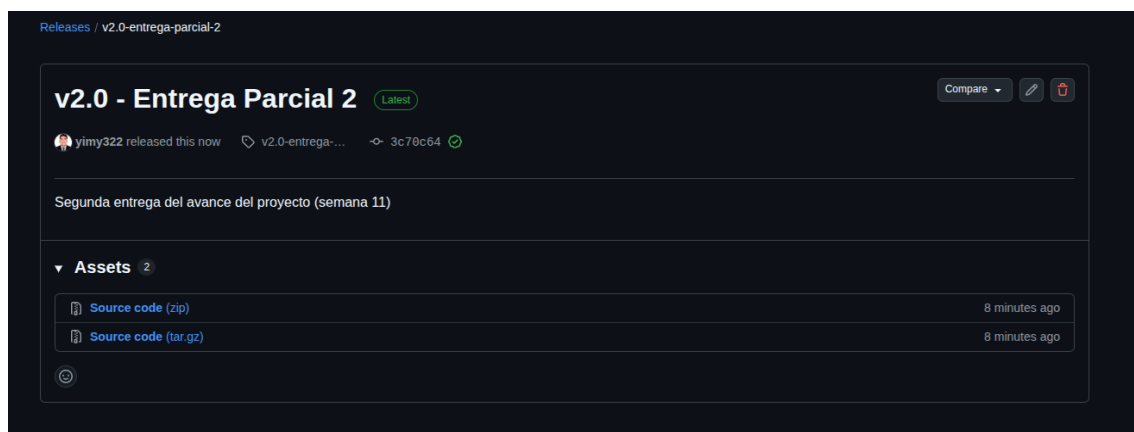
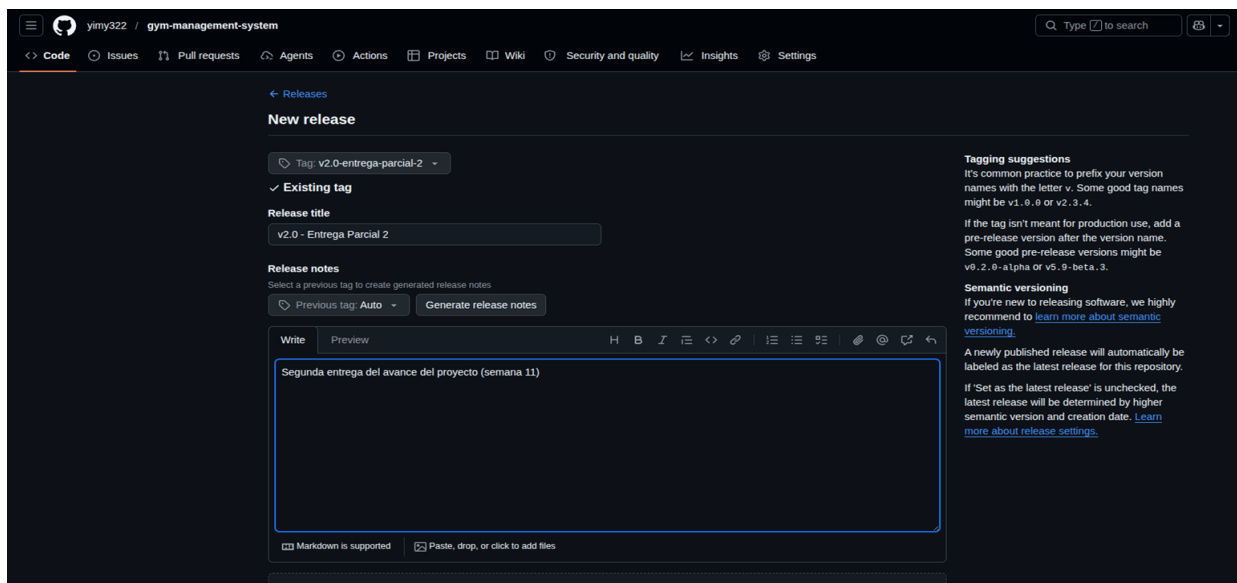
The screenshot displays a GitHub Pull Request interface. At the top, the title is "Release/v2.0 entrega parcial 2 #55". Below the title, it indicates the pull request is "Merged" and shows the commit history. The main content area lists the commits, including "Feat: configurar bootstrap - STRK-16 #done", "Merge pull request #37 from yimy322/feature/STRK-16-configurar-bootstrap", "Feat: crear fragmento navbar - STRK-3", "chore: eliminar fragments.html obsoleto - STRK-3", "Feat: agregar calendario de asistencias en panel lateral - STRK-6 #done", "Feat: maquetar pantalla de control de asistencia - STRK-10 #done", "Feat: agregar estilos y actualizar navbar", "Feat: dependencias: implementar crud de tipos de membresia - STRK-14", "Merge pull request #38 from yimy322/feature/STRK-6-calendario-asisten", "Feat: maquetar dashboard con tarjetas metricas - STRK-3 #done", and "Merge pull request #41 from yimy322/feature/STRK-3-dashboard-general". The right sidebar contains sections for "Reviewers" (yxdhii), "Assignees" (No one—assign yourself), "Labels" (None yet), "Projects" (None yet), "Milestone" (No milestone), "Development" (Successfully merging this pull request may close these issues), and "Notifications" (Unsubscribe).

Paso 5. Con el merge aprobado, se crea el tag y se sube al repositorio remoto.

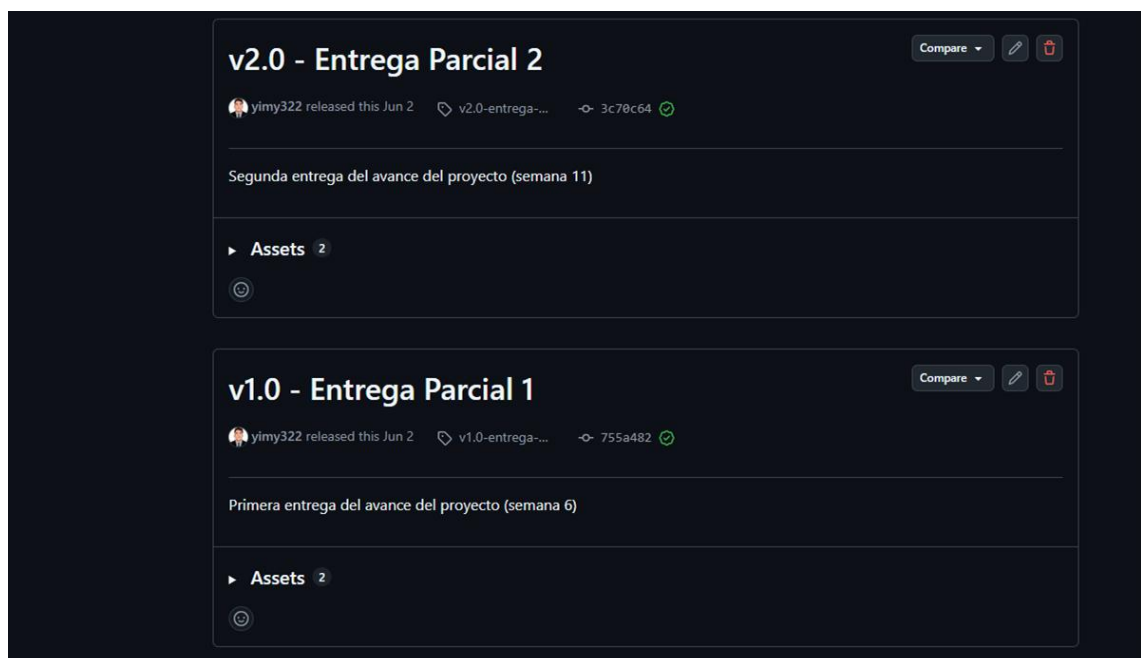
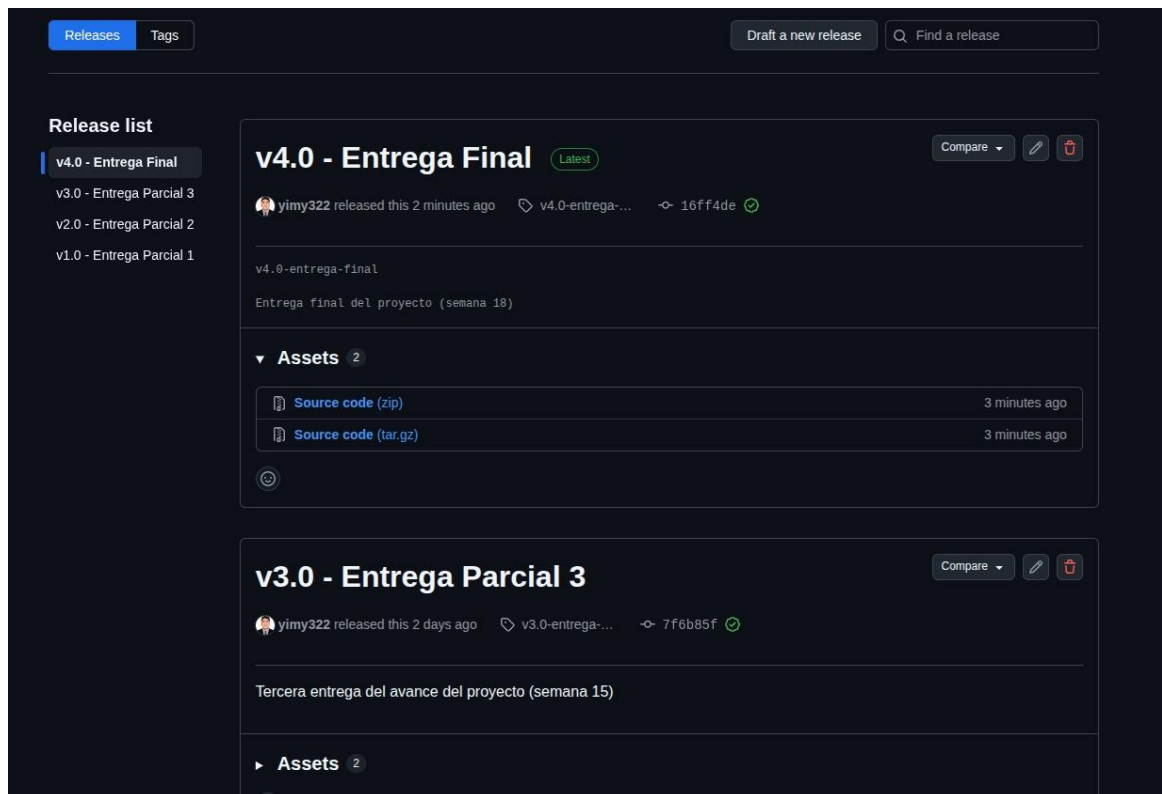
```
Yadhira Saavedra@LAPTOP-22GAEEESQ MINGW64 ~/Documents/STRK
$ git tag -a v2.0-entrega-parcial-2 -m "Avance 2 completo"
$ git push origin v2.0-entrega-parcial-2
```

```
yiny@PC-322:~/universidad/ciclo-8-prt2/herramientas-desarrollo/gym-management-system$ git tag -a v2.0-entrega-parcial-2 -m "Avance 2 completo"
yiny@PC-322:~/universidad/ciclo-8-prt2/herramientas-desarrollo/gym-management-system$ git push origin v2.0-entrega-parcial-2
Username for 'https://github.com': yiny322
Password for 'https://yiny322@github.com':
Enumerating objects: 1, done.
Counting objects: 100% (1/1), done.
Writing objects: 100% (1/1), 172 bytes | 172.00 KiB/s, done.
Total 1 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com:yiny322/gym-management-system.git
 * [new tag]          v2.0-entrega-parcial-2 -> v2.0-entrega-parcial-2
yiny@PC-322:~/universidad/ciclo-8-prt2/herramientas-desarrollo/gym-management-system$
```

Paso 6. En GitHub se crea el release seleccionando el tag v2.0-entrega-parcial-2, generando el código fuente descargable en .zip y .tar.gz.



Posteriormente, se publicaron los demás releases del proyecto en GitHub, completando el historial de versiones correspondientes a cada entrega.



** Vista de la sección Releases del repositorio de GitHub, donde se muestran las versiones publicadas del proyecto (v1.0, v2.0, v3.0 y v4.0), correspondientes a las entregas realizadas durante su desarrollo.*

4.13. Pipeline CI/CD y DevSecOps

4.13.1. Herramientas integradas en el pipeline

El proyecto implementa un pipeline de Integración Continua y DevSecOps mediante GitHub Actions, el cual se ejecuta automáticamente ante cada push a la rama develop. Este pipeline integra pruebas unitarias, análisis de calidad de código, análisis de seguridad de dependencias, construcción de la imagen Docker, escaneo de vulnerabilidades de la imagen y despliegue en la nube, combinando en un solo flujo las prácticas de CI (Integración Continua) y DevSecOps descritas en el marco teórico.

4.13.2. Diagrama del pipeline

El flujo completo del pipeline se ejecuta en el siguiente orden:

Paso 1	Push a la rama de develop, lo cual dispara la ejecución automática del pipeline.
Paso 2	Descarga del código fuente del repositorio (checkout).
Paso 3	Configuración del entorno de ejecución con Java 17.
Paso 4	Compilación del proyecto y ejecución de las pruebas unitarias con Junit, generando el reporte de cobertura con JaCoCo.
Paso 5	Análisis estático de calidad de código mediante SonarCloud.
Paso 6	Análisis de vulnerabilidades en las dependencias del proyecto mediante Snyk.
Paso 7	Construcción (build) de la imagen Docker de la aplicación.
Paso 8	Escaneo de vulnerabilidades del sistema operativo de la imagen mediante Trivy.
Paso 9	Autenticación (Login) en Docker Hub.
Paso 10	Publicación (push) de la imagen Docker en Docker Hub, quedando disponible para su despliegue en Railway.

DIAGRAMA DEL PIPELINE



Archivo de configuración del pipeline en GitHub Actions, activado ante cada push o pull request a main o develop. Define un job (build-and-test) sobre ubuntu-latest con nueve pasos secuenciales: descarga de código, configuración de Java 17, pruebas con JaCoCo, análisis con SonarCloud, análisis con Snyk, build de Docker, escaneo con Trivy, login y push a Docker Hub. Las credenciales se manejan mediante secrets de GitHub.

Representación gráfica de las nueve etapas del pipeline, desde el push a develop hasta la publicación de la imagen en Docker Hub, mostrando de forma visual la integración de prácticas de CI (pruebas) y DevSecOps (análisis de calidad y seguridad).

```

1  name: CI Pipeline - Gym Management System
2
3  on:
4    push:
5      branches:
6        - main
7        - develop
8
9    pull request:
10     branches:
11       - main
12       - develop
13
14  jobs:
15    build-and-test:
16
17     runs-on: ubuntu-latest
18
19     steps:
20
21     - name: 1. Descargar Código Fuente
22       uses: actions/checkout@v4
23       with:
24         fetch-depth: 0
25
26     - name: 2. Configurar Entorno Java 17
27       uses: actions/setup-java@v4
28       with:
29         distribution: temurin
30         java-version: 17
31
32     - name: 3. Compilación y Pruebas Unitarias (JaCoCo Coverage)
33       run: mvn clean verify
34
35     - name: 4. Análisis de Calidad de Código (SonarCloud)
36       env:
37         SONAR_TOKEN: ${ secrets.SONAR_TOKEN }
38       run: |
39         mvn sonar:sonar \
40           -Dsonar.projectKey=yimy322_gym-management-system2 \
41           -Dsonar.organization=yimy322 \
42           -Dsonar.host.url=https://sonarcloud.io \
43           -Dsonar.coverage.jacoco.xmlReportPaths=target/site/jacoco/jacoco.xml
44
45     - name: 5. Análisis de Seguridad de Dependencias (Snyk SAST)
46       env:
47         SNYK_TOKEN: ${ secrets.SNYK_TOKEN }
48       run: |
49         npm install -g snyk
50         snyk test || true
51         #el test escanea el pom
52         #el true significa que si hay vulnerabi... rompera el ci
53
54     - name: 6. Empaquetado - Construir Imagen Docker
55       run: docker build -t ${ secrets.DOCKERHUB_USERNAME }}/gym-app:latest .
56
57     - name: 7. Escaneo de Vulnerabilidades de Sistema Operativo (Trivy)
58       uses: aquasecurity/trivy-action@master
59       with:
60         image-ref: ${ secrets.DOCKERHUB_USERNAME }}/gym-app:latest
61         format: 'table'
62         exit-code: '0' #no rompera el ci
63         severity: 'HIGH,CRITICAL'
64
65     - name: 8. Login a Docker Hub
66       uses: docker/login-action@v3
67       with:
68         username: ${ secrets.DOCKERHUB_USERNAME }
69         password: ${ secrets.DOCKERHUB_TOKEN }
70
71     - name: 9. Despliegue - Subir contenedor a Docker Hub
72       run: docker push ${ secrets.DOCKERHUB_USERNAME }}/gym-app:latest
  
```

4.13.3. Pruebas unitarias (JUnit + Mockito)

Para garantizar la correcta lógica de negocio del sistema, se implementaron pruebas unitarias sobre cuatro servicios principales de la aplicación, ubicados en *src/test/java/com.gymmanagement.gym.services*, utilizando el framework JUnit 5 en conjunto con Mockito para la simulación (mocking) de dependencias:

```
@ExtendWith(MockitoExtension.class)
public class MemberServiceTest {

    @Mock
    private MemberRepository memberRepository;

    @InjectMocks
    private MemberServiceImpl memberService;

    private Member member;

    @BeforeEach
    void setUp() {
        member = new Member();
        member.setId(id: 1L);
        member.setFirstName(firstName: "Juan");
        member.setLastName(lastName: "Perez");
        member.setDni(dni: "12345678");
    }

    //CREATE
    @Test
    void save_shouldSaveMember() {
        when(memberRepository.save(member)).thenReturn(member);
        Member result = memberService.save(member);
        assertNotNull(result);
        assertEquals(member.getId(), result.getId());
        verify(memberRepository, times(wantedNumberOfInvocations: 1)).save(member);
    }

    //READ ALL
    @Test
    void findAll_shouldReturnMemberList() {
        when(memberRepository.findAll()).thenReturn(List.of(member));
        List<Member> result = memberService.findAll();
    }
}
```

Estructura de las clases de prueba en el proyecto (AttendanceServiceTest, MemberServiceTest, MembershipsServiceTest, SubscriptionServiceTest).

La distribución de pruebas por servicio es la siguiente

Servicio	Nº de pruebas
AttendanceService	6
MemberService	5
MembershipService	6
SubscriptionService	5
Total	22

```
src/test/java
└─ com.gymmanagement.gym.services
   ├── AttendanceServiceTest
   ├── MemberServiceTest
   ├── MembershipsServiceTest
   └── SubscriptionServiceTest
```

Al ejecutar el comando *mvn clean verify*, el proyecto compila correctamente y las cuatro clases de prueba se ejecutan sin fallos, obteniéndose un resultado de **22 pruebas ejecutadas, 0 fallos, 0 errores y 0 omitidas** (Tests run: 22, Failures: 0, Errors: 0, Skipped: 0), lo cual confirma que la lógica de negocio del sistema cumple con el comportamiento esperado antes de avanzar a las siguientes etapas del pipeline (análisis de cobertura con JaCoCo y análisis de calidad con SonarCloud).

```
yimy@PC-322:~/universidad/ciclo-8-prt2/herramientas-desarrollo/gym-management-system$ mvn clean verify
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.gymmanagement.gym.services.MembershipsServiceTest
OpenJDK 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.350 s -- in com.gymmanagement.gym.services.MembershipsServiceTest
[INFO] Running com.gymmanagement.gym.services.MemberServiceTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.097 s -- in com.gymmanagement.gym.services.MemberServiceTest
[INFO] Running com.gymmanagement.gym.services.AttendanceServiceTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.073 s -- in com.gymmanagement.gym.services.AttendanceServiceTest
[INFO] Running com.gymmanagement.gym.services.SubscriptionServiceTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.067 s -- in com.gymmanagement.gym.services.SubscriptionServiceTest
[INFO] Results:
[INFO] Tests run: 22, Failures: 0, Errors: 0, Skipped: 0
[INFO]
```

** Resultado de la ejecución de mvn clean verify en consola, mostrando el detalle de cada clase de prueba y el conteo final de 22 pruebas exitosas.*

Finalmente, el proceso culmina con un resultado **BUILD SUCCESS**, confirmando que tanto la compilación como la totalidad de las pruebas unitarias se completaron correctamente, en un tiempo total de ejecución de 6.915 segundos.

```
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 6.915 s
[INFO] Finished at: 2026-06-23T08:23:29-05:00
[INFO] -----
```

** Resultado final de la ejecución (BUILD SUCCESS) con el tiempo total del proceso.*

Este resultado confirma que la lógica de negocio del sistema cumple con el comportamiento esperado antes de avanzar a las siguientes etapas del pipeline (análisis de cobertura con JaCoCo y análisis de calidad con SonarCloud).

4.13.4. JaCoCo – Medición de cobertura de pruebas

JaCoCo (Java Code Coverage) es un plugin de Maven que mide qué porcentaje del código fuente es efectivamente ejecutado durante las pruebas unitarias. Dentro del proyecto, JaCoCo se integra en el *pom.xml* mediante dos ejecuciones (goals) dentro del ciclo de vida de Maven:

- **prepare-agent:** se engancha antes de ejecutar las pruebas y registra en tiempo de ejecución qué líneas y ramas del código son recorridas.
- **report** (fase verify): una vez finalizadas las pruebas, genera un reporte detallado en [target/site/jacoco/jacoco.xml](#) y [target/site/jacoco/index.html](#).

```

<plugin>
  <groupId>org.jacoco</groupId>
  <artifactId>jacoco-maven-plugin</artifactId>
  <version>0.8.11</version>
  <executions>
    <execution>
      <goals><goal>prepare-agent</goal></goals>
    </execution>
    <execution>
      <id>report</id>
      <phase>verify</phase>
      <goals>
        <goal>report</goal>
      </goals>
    </execution>
  </executions>
</plugin>

```

** Configuración del plugin JaCoCo en el pom.xml*

Este reporte se genera automáticamente al ejecutar *mvn clean verify* y cumple dos funciones dentro del pipeline:

- Es consumido por SonarCloud mediante el parámetro - *Dsonar.coverage.jacoco.xmlReportPaths=target/site/jacoco/jacoco.xml*, que lo lee para mostrar la cobertura en su dashboard.
- Permite identificar con precisión qué clases y métodos del proyecto no cuentan con pruebas, orientando futuras mejoras en la suite de testing.

El reporte muestra una cobertura global de 39% sobre el total del proyecto (465 de 763 instrucciones cubiertas, 40% de cobertura de ramas). El detalle por paquete evidencia que la capa de lógica de negocio (**services.impl**) concentra la mayor cobertura con un 71%, mientras que las capas de controladores (**controllers**) y entidades (**entities**) presentan 0% de cobertura, al no contar aún con pruebas unitarias o de integración dedicadas. Esto confirma que el esfuerzo de testing del proyecto se concentró en la capa de servicios, que es donde reside la lógica de negocio crítica del sistema.

 gym-management-system

gym-management-system

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.gymmanagement.gym.controllers		0%		0%	28	28	65	65	25	25	7	7
com.gymmanagement.gym.services.impl		71%		57%	15	38	18	88	11	31	2	6
com.gymmanagement.gym		0%		n/a	7	7	20	20	7	7	2	2
com.gymmanagement.gym.entities		0%		n/a	4	4	9	9	4	4	3	3
com.gymmanagement.gym.utils		100%		n/a	0	1	0	3	0	1	0	1
Total	465 of 763	39%	12 of 20	40%	54	78	112	185	47	68	14	19

** Reporte HTML de JaCoCo (target/site/jacoco/index.html) con el desglose de cobertura por paquete.*

Posteriormente, tras la implementación de nuevas pruebas unitarias e integración, se generó un reporte actualizado de cobertura. Como resultado, la cobertura global aumentó al 99% de instrucciones (1 245 de 1 247 instrucciones cubiertas) y 97% de cobertura de ramas (45 de 46 ramas cubiertas). Asimismo, los paquetes controllers, mapper y entities alcanzaron una cobertura del 100%, mientras que services.impl obtuvo un 99% de cobertura de instrucciones y un 90% de cobertura de ramas, evidenciando una mejora significativa en la validación de los componentes del sistema.

gym-management-system

gym-management-system

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.gymmanagement.gym.services.impl		99%		90%	1	46	0	92	0	41	0	5
com.gymmanagement.gym.controllers		100%		100%	0	48	0	137	0	30	0	6
com.gymmanagement.gym.mapper		100%		n/a	0	4	0	16	0	4	0	2
com.gymmanagement.gym.entities		100%		n/a	0	4	0	9	0	4	0	3
Total	2 of 1,247	99%	1 of 46	97%	1	102	0	254	0	79	0	16

** Reporte HTML de JaCoCo (target/site/jacoco/index.html) actualizado con el desglose de cobertura por paquete.*

4.13.5. SonarCloud – Análisis de calidad de código

SonarCloud es una plataforma en la nube que realiza análisis estático del código fuente, examinando el código sin necesidad de ejecutarlo, e identificando problemas de calidad, mantenibilidad y seguridad. En el proyecto, SonarCloud evalúa las siguientes categorías:

Categoría	Qué detecta
Bugs	Errores en el código que pueden provocar fallos en tiempo de ejecución.

Vulnerabilidades	Problemas de seguridad explotables dentro del propio código.
Code Smells	Código que funciona correctamente, pero está mal estructurado o es difícil de mantener.
Cobertura	Porcentaje de código cubierto por pruebas unitarias, obtenido a partir del reporte de JaCoCo.
Duplicación	Bloques de código repetidos innecesariamente dentro del proyecto.

❖ *Integración al pipeline*

El análisis se ejecuta como un step de GitHub Actions llamado "Análisis de Calidad de Código (SonarCloud)", que utiliza el token almacenado en secrets.SONAR_TOKEN y ejecuta el comando `mvn sonar:sonar` apuntando al proyecto (`yimy322_gym-management-system2`), la organización (`yimy322`) y el reporte de cobertura generado previamente por JaCoCo.

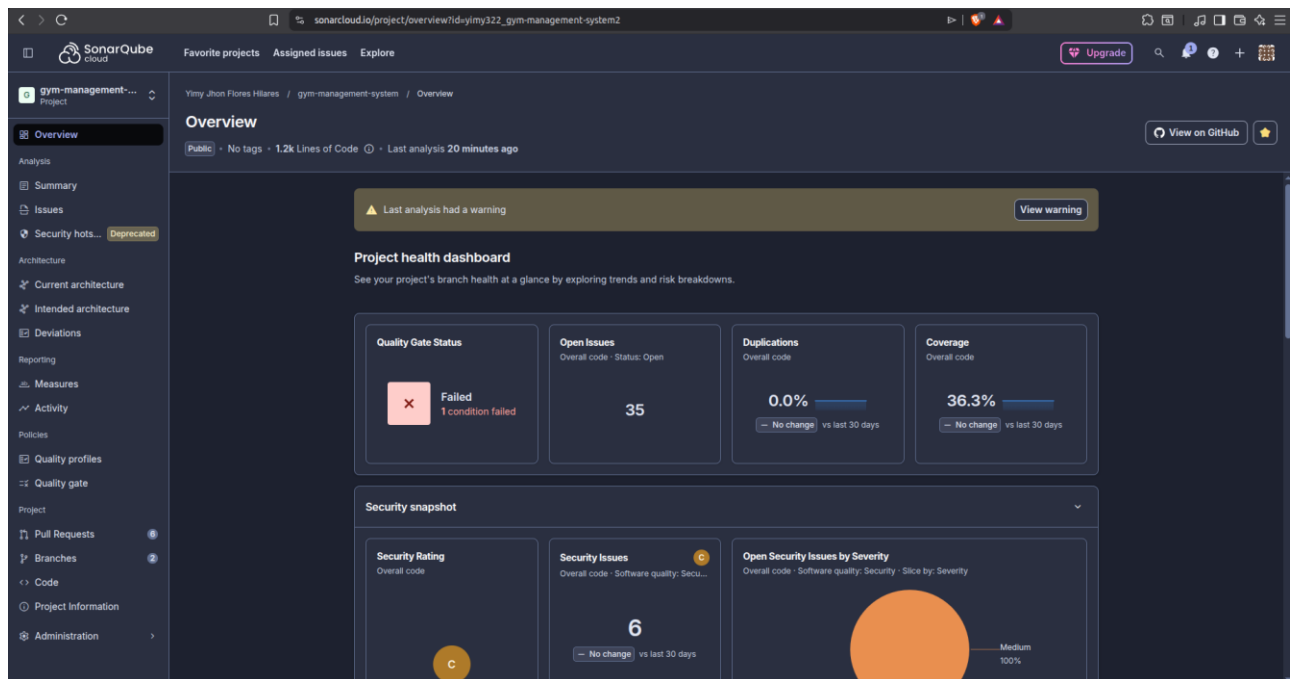
```
- name: 4. Analisis de Calidad de Codigo (SonarCloud)
  env:
    | SONAR_TOKEN: ${ secrets.SONAR_TOKEN }
  run: |
    | mvn sonar:sonar \
    |   -Dsonar.projectKey=yimy322_gym-management-system2 \
    |   -Dsonar.organization=yimy322 \
    |   -Dsonar.host.url=https://sonarcloud.io \
    |   -Dsonar.coverage.jacoco.xmlReportPaths=target/site/jacoco/jacoco.xml
```

** Step de GitHub Actions que ejecuta el análisis de SonarCloud.*

❖ *Resultados obtenidos en el proyecto*

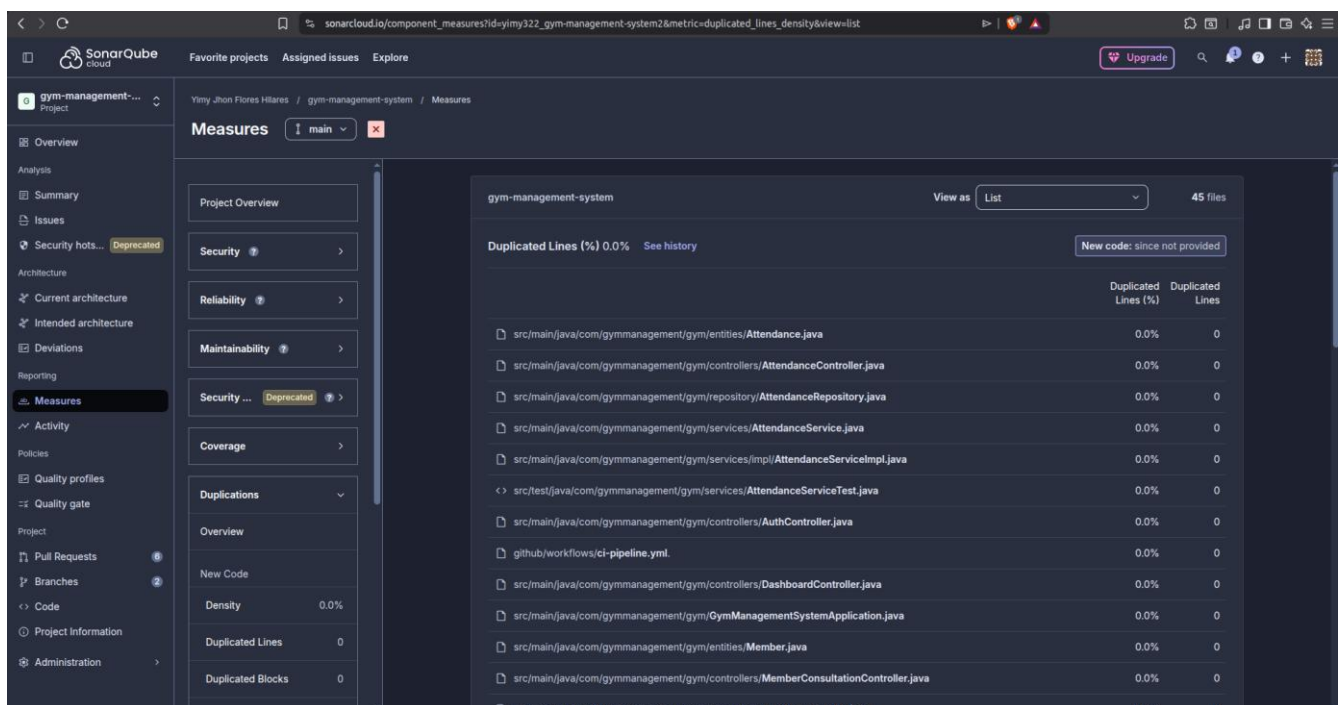
El dashboard de SonarCloud refleja, sobre un total de 1.2k líneas de código:

- Quality Gate Status: Failed (1 condición no cumplida).
- Cobertura general: 36.3%.
- Duplicación de código: 0.0%.
- Security Rating: C, con 6 problemas de seguridad abiertos (severidad media en su mayoría).
- 35 issues abiertos en el código general.



*** Dashboard general (Overview) del proyecto en SonarCloud.**

Al revisar la sección de Measures, se confirma que la duplicación de código se mantiene en 0.0% en todos los archivos analizados (45 archivos en total), lo que indica buenas prácticas de reutilización de código mediante servicios, mappers y controladores bien diferenciados.



*** Medición de duplicación de líneas por archivo en SonarCloud.**

En cuanto a cobertura, el detalle por archivo muestra que clases como *Attendance.java*, *MemberMapper.java* o *AttendanceController.java* presentan 0.0% de cobertura individual, ya que las pruebas unitarias actuales se enfocan en la capa de servicios y no en controladores, entidades o mappers, replicando el mismo patrón observado en el reporte de JaCoCo.

gym-management-system			
Coverage 36.3%		Uncovered Lines	Uncovered Conditions
src/main/java/com/gymmanagement/gym/entities/Attendance.java	0.0%	2	-
src/main/java/com/gymmanagement/gym/controllers/AttendanceController.java	0.0%	12	-
src/main/java/com/gymmanagement/gym/controllers/AuthController.java	0.0%	2	-
src/main/java/com/gymmanagement/gym/controllers/DashboardController.java	0.0%	5	-
src/main/java/com/gymmanagement/gym/GymManagementSystemApplication.java	0.0%	3	-
src/main/java/com/gymmanagement/gym/entities/Member.java	0.0%	2	-
src/main/java/com/gymmanagement/gym/controllers/MemberConsultationController.java	0.0%	2	-
src/main/java/com/gymmanagement/gym/controllers/MemberController.java	0.0%	26	6
src/main/java/com/gymmanagement/gym/mapper/MemberMapper.java	0.0%	9	-
src/main/java/com/gymmanagement/gym/controllers/MembershipController.java	0.0%	7	-
src/main/java/com/gymmanagement/gym/mapper/MembershipMapper.java	0.0%	8	-
src/main/java/com/gymmanagement/gym/services/impl/PaymentServiceImpl.java	0.0%	2	2
src/main/java/com/gymmanagement/gym/SecurityConfig.java	0.0%	17	-

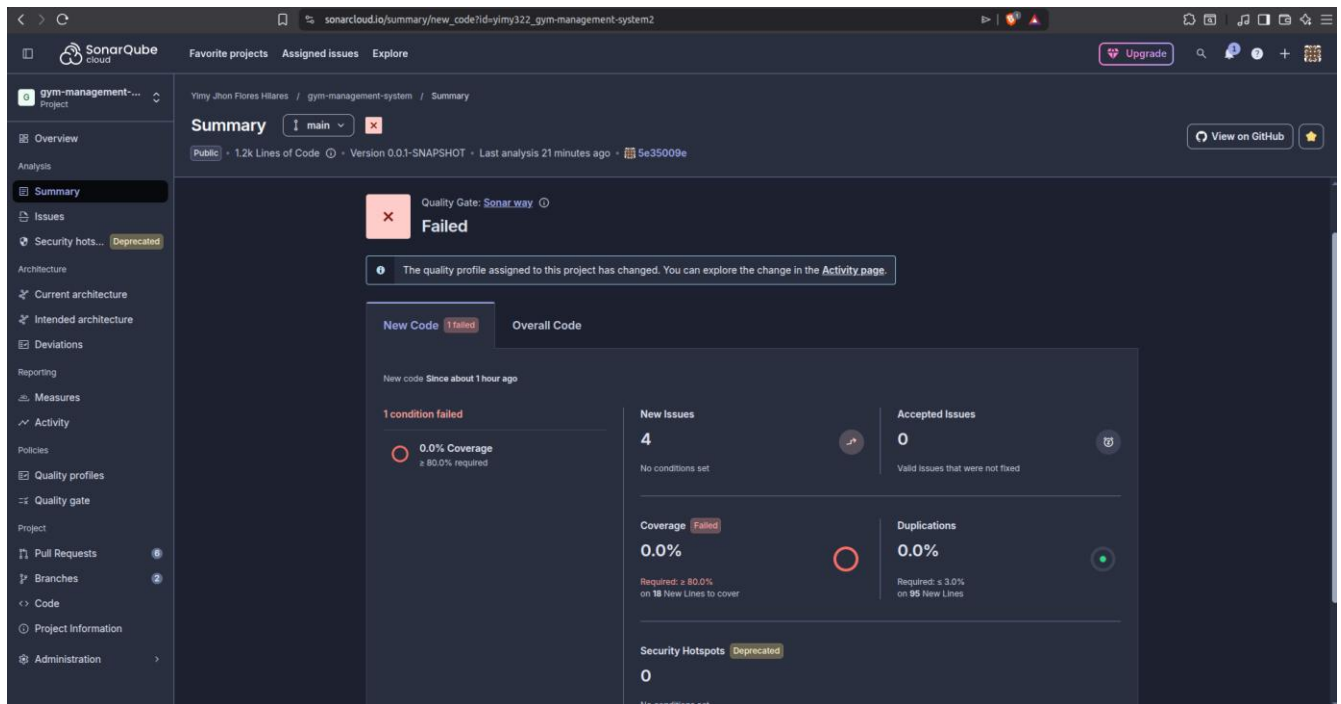
* *Medición de cobertura por archivo en SonarCloud (cobertura total: 36.3%).*

❖ *Quality Gate*

El Quality Gate es un umbral de calidad que SonarCloud evalúa automáticamente tras cada análisis:

- Si el código no cumple los estándares mínimos → el resultado es **FAILED**.
- Si los cumple → el resultado es **PASSED**.

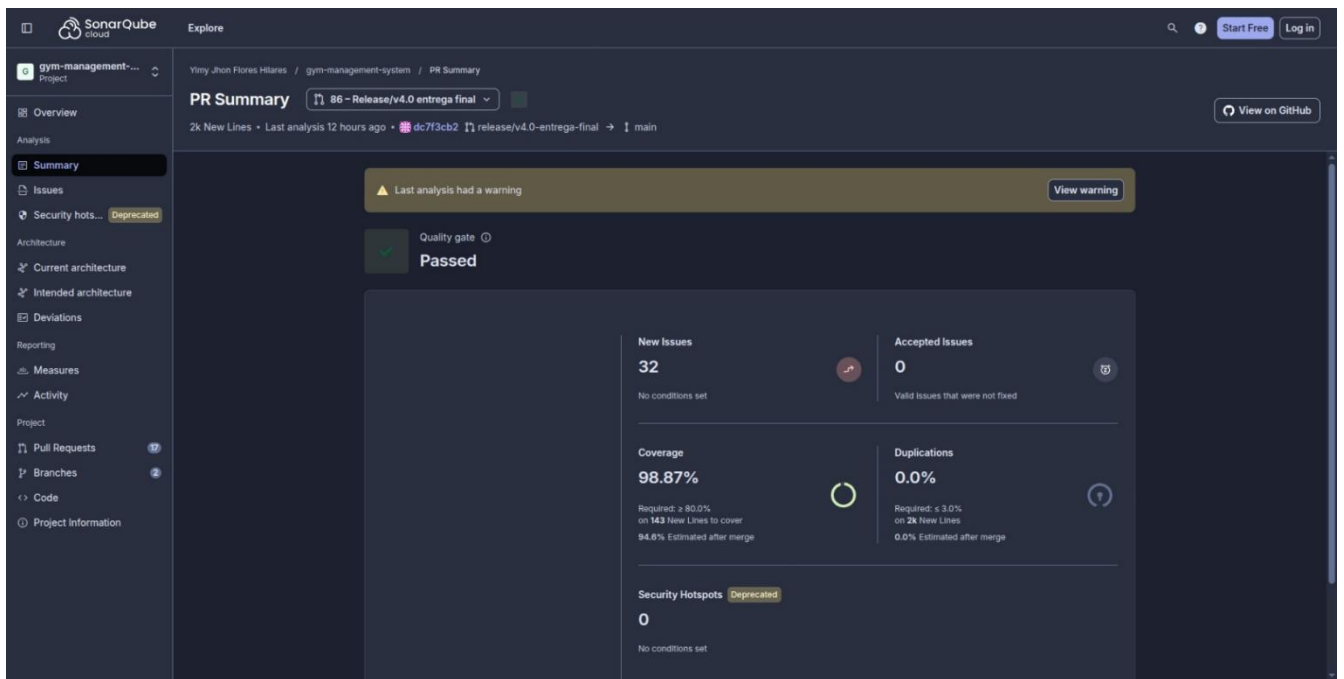
En el caso del proyecto, el resumen de *New Code* muestra un Quality Gate en estado **Failed**, debido a que la condición de cobertura mínima ($\geq 80.0\%$ requerida) no se cumple, registrándose apenas 0.0% de cobertura sobre las 18 líneas nuevas evaluadas, junto con 4 issues nuevos detectados. Las demás condiciones, como duplicación (0.0%, dentro del límite de $\leq 3.0\%$), sí se cumplen.



**** Resumen del Quality Gate marcado como Failed por incumplimiento de la condición de cobertura.***

Este resultado es coherente con el análisis de JaCoCo: al concentrarse las pruebas unitarias únicamente en la capa de servicios, el código nuevo agregado en controladores y otras capas no alcanza el porcentaje de cobertura exigido por el Quality Gate, quedando como una oportunidad de mejora identificada para futuras iteraciones del proyecto.

Posteriormente, se implementaron pruebas unitarias adicionales para incrementar la cobertura del código nuevo. Como resultado, el proyecto alcanzó una cobertura del 98.87%, sin duplicación de código (0.0%), lo que permitió que SonarCloud validara satisfactoriamente todos los criterios establecidos y el Quality Gate cambiara a estado Passed.



*** Resultado final del Quality Gate en estado Passed, evidenciando el cumplimiento de los criterios de calidad establecidos por SonarCloud.**

Este resultado es coherente con la medición de cobertura obtenida mediante JaCoCo, donde se alcanzó un 99% de cobertura de instrucciones y un 97% de cobertura de ramas, permitiendo cumplir el umbral mínimo de cobertura exigido por el Quality Gate de SonarCloud y contribuyendo a la aprobación del análisis de calidad.

4.13.6. Snyk y Trivy - Análisis de seguridad

¿Por qué analizar seguridad en el pipeline?

Las dependencias utilizadas en el proyecto (Spring Boot, MySQL, etc.) pueden contener vulnerabilidades conocidas, denominadas CVE (Common Vulnerabilities and Exposures). Asimismo, una imagen Docker puede incluir librerías del sistema operativo desactualizadas y vulnerables, incluso si el código de la aplicación está libre de fallos. Por ello, el pipeline incorpora dos herramientas complementarias de análisis de seguridad: Snyk, enfocada en las dependencias de la aplicación, y Trivy, enfocada en la imagen Docker.

❖ Snyk

Snyk escanea el archivo **pom.xml** del proyecto en busca de dependencias con vulnerabilidades conocidas (CVE). Según el diagrama del pipeline este análisis corresponde al paso "5. *Análisis de Seguridad de Dependencias (Snyk SAST)*":

```
- name: 5. Analisis de Seguridad de Dependencias (Snyk SAST)
  env:
    | SNYK_TOKEN: ${ secrets.SNYK_TOKEN }
  run: |
    | npm install -g snyk
    | snyk test || true
```

** Step de GitHub Actions correspondiente al análisis de Snyk.*

El proceso instala Snyk de forma global (npm install -g snyk) y ejecuta snyk test, autenticándose mediante el token secrets.SNYK_TOKEN. El comando se ejecuta con la bandera || true, por lo que, si se detectan vulnerabilidades, el análisis queda registrado de forma informativa sin detener la ejecución del pipeline.

❖ Trivy

Trivy escanea la imagen Docker ya construida, buscando vulnerabilidades a nivel de sistema operativo y paquetes base. A diferencia de Snyk, que revisa las dependencias Java del proyecto, Trivy analiza el contenedor completo. Según el diagrama del pipeline este análisis corresponde al paso "7. *Escaneo de Vulnerabilidades de Sistema Operativo (Trivy)*":

```
- name: 7. Escaneo de Vulnerabilidades de Sistema Operativo (Trivy)
  uses: aquasecurity/trivy-action@master
  with:
    | image-ref: '${ secrets.DOCKERHUB_USERNAME }/gym-app:latest'
    | format: 'table'
    | exit-code: '0'
    | severity: 'HIGH,CRITICAL'
```

** Step de GitHub Actions correspondiente al análisis de Trivy.*

El proceso utiliza la acción oficial *aquasecurity/trivy-action@master*, configurada para analizar la imagen `{{ secrets.DOCKERHUB_USERNAME }}/gym-app:latest`, mostrando los resultados en formato table y filtrando únicamente vulnerabilidades de severidad HIGH y CRITICAL. Al igual que en Snyk, se define exit-code: '0', por lo que el pipeline no se detiene, aunque se detecten vulnerabilidades; el escaneo cumple una función informativa para el equipo de desarrollo.

4.13.7. Resultados del escaneo de Snyk

El escaneo de Snyk sobre el *pom.xml* del proyecto identificó 30 issues en total (1 crítico, 13 altos, 15 medios, 1 bajo), concentrados principalmente en dependencias de **Spring Boot 3.5.14** como *spring-boot-starter-data-jpa*, *spring-boot-starter-security* y *spring-boot-devtools*.

The screenshot shows the Snyk web interface for a project named 'yimy322/gym-management-system'. The main view displays the results of a scan on the 'pom.xml' file. The interface is divided into a left sidebar with navigation options (Dashboard, Projects, Integrations, Members, Settings) and a main content area. The main content area shows two sections of vulnerabilities, each with a 'Priority Score (MAX)'.

Section 1: org.springframework.boot:spring-boot-starter-data-jpa@3.5.14
 Priority Score (MAX): 721
 10 transitive issues
 Fixable issues: 10 | Issues with no supported fix: 0 | Vulnerable dependencies: 0
 Upgrading to org.springframework.boot:spring-boot-starter-data-jpa@3.5.15 fixes 10 issues
 Issues listed:
 - Inefficient Algorithmic Complexity (org.springframework:spring-expression@6.2.18) - CWE-407, CVSS 8.7, 721
 - Allocation of Resources Without Limits or Throttling (org.springframework.data:spring-data-commons@3.5.11) - CWE-770, CVSS 8.7, 721
 - Denial of Service (DoS) (org.springframework.data:spring-data-commons@3.5.11) - CWE-400, CVSS 8.7, 721
 Button: Upgrade to 3.5.15

Section 2: org.springframework.boot:spring-boot-devtools@3.5.14
 Priority Score (MAX): 696
 4 transitive issues
 Fixable issues: 4 | Issues with no supported fix: 0 | Vulnerable dependencies: 0
 Upgrading to org.springframework.boot:spring-boot-devtools@3.5.15 fixes 4 issues
 Issues listed:
 - Allocation of Resources Without Limits or Throttling (org.springframework:spring-core@6.2.18) - CWE-770, CVSS 8.2, 696
 - Regular Expression Denial of Service (ReDoS) (org.springframework:spring-core@6.2.18) - CWE-1333, CVSS 6.3, 601
 - Insecure Temporary File (org.springframework.boot:spring-boot-autoconfigure@3.5.14) - CWE-377, CVSS 4.8, 526
 Button: Show more issues

* *Vulnerabilidades detectadas por Snyk en pom.xml, agrupadas por dependencia.*

** Vista general de severidad y priorización de las 30 vulnerabilidades encontradas*

Snyk reporta que la totalidad de los issues son corregibles (fixable); todo se arregla actualizando a la versión Spring Boot 3.5.15.

4.13.8. Secrets en GitHub

Los *secrets* son variables cifradas que GitHub almacena de forma segura a nivel de repositorio. El pipeline puede utilizarlas sin que nadie pueda ver su valor real, evitando exponer credenciales sensibles en el código del workflow ni en los logs de ejecución.

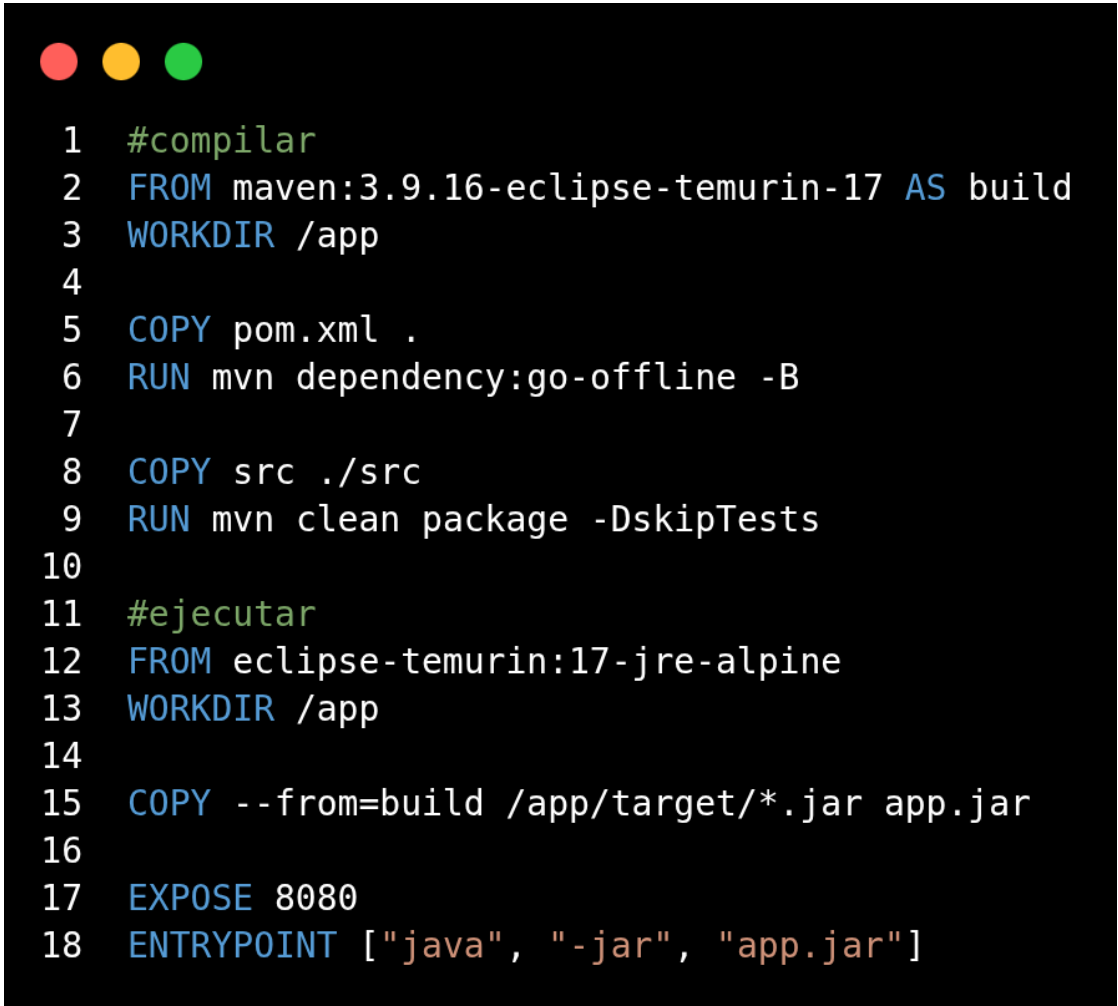
** Secrets configurados en el repositorio: DOCKERHUB_TOKEN, DOCKERHUB_USERNAME, SNYK_TOKEN, SONAR_TOKEN.*

4.13.9. Dockerfile multi-etapa

La imagen Docker del proyecto se construye mediante un Dockerfile multi-etapa (*multi-stage build*):

- Etapa 1 (build): imagen Maven que compila el proyecto y genera el .jar.
- Etapa 2 (runtime): imagen liviana JRE Alpine que solo ejecuta el .jar.

Esta estrategia evita que las herramientas de compilación queden incluidas en la imagen final, reduciendo su tamaño y su superficie de ataque.

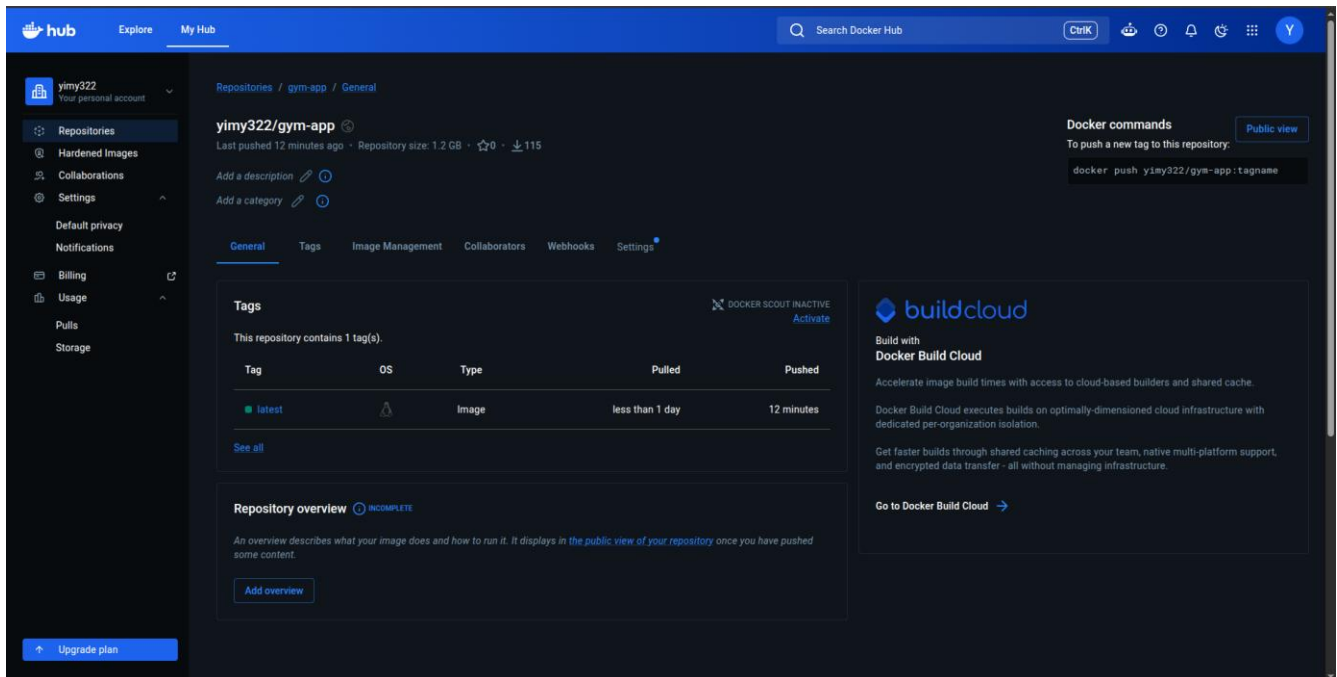
A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. The terminal displays a Dockerfile with 18 lines of code, numbered 1 through 18. The code is organized into two stages: a build stage (lines 1-9) and a runtime stage (lines 11-18). The build stage starts with a comment '#compilar', followed by 'FROM maven:3.9.16-eclipse-temurin-17 AS build', 'WORKDIR /app', and then 'COPY pom.xml .', 'RUN mvn dependency:go-offline -B', and 'COPY src ./src'. The runtime stage starts with a comment '#ejecutar', followed by 'FROM eclipse-temurin:17-jre-alpine', 'WORKDIR /app', 'COPY --from=build /app/target/*.jar app.jar', 'EXPOSE 8080', and 'ENTRYPOINT ["java", "-jar", "app.jar"]'.

```
1  #compilar
2  FROM maven:3.9.16-eclipse-temurin-17 AS build
3  WORKDIR /app
4
5  COPY pom.xml .
6  RUN mvn dependency:go-offline -B
7
8  COPY src ./src
9  RUN mvn clean package -DskipTests
10
11 #ejecutar
12 FROM eclipse-temurin:17-jre-alpine
13 WORKDIR /app
14
15 COPY --from=build /app/target/*.jar app.jar
16
17 EXPOSE 8080
18 ENTRYPOINT ["java", "-jar", "app.jar"]
```

** Dockerfile multi-etapa del proyecto.*

4.13.10. Docker Hub

Una vez construida y escaneada, la imagen se publica en el repositorio yimy322/gym-app de Docker Hub, bajo el tag latest, quedando disponible para su despliegue.



** Repositorio gym-app en Docker Hub con la imagen publicada.*

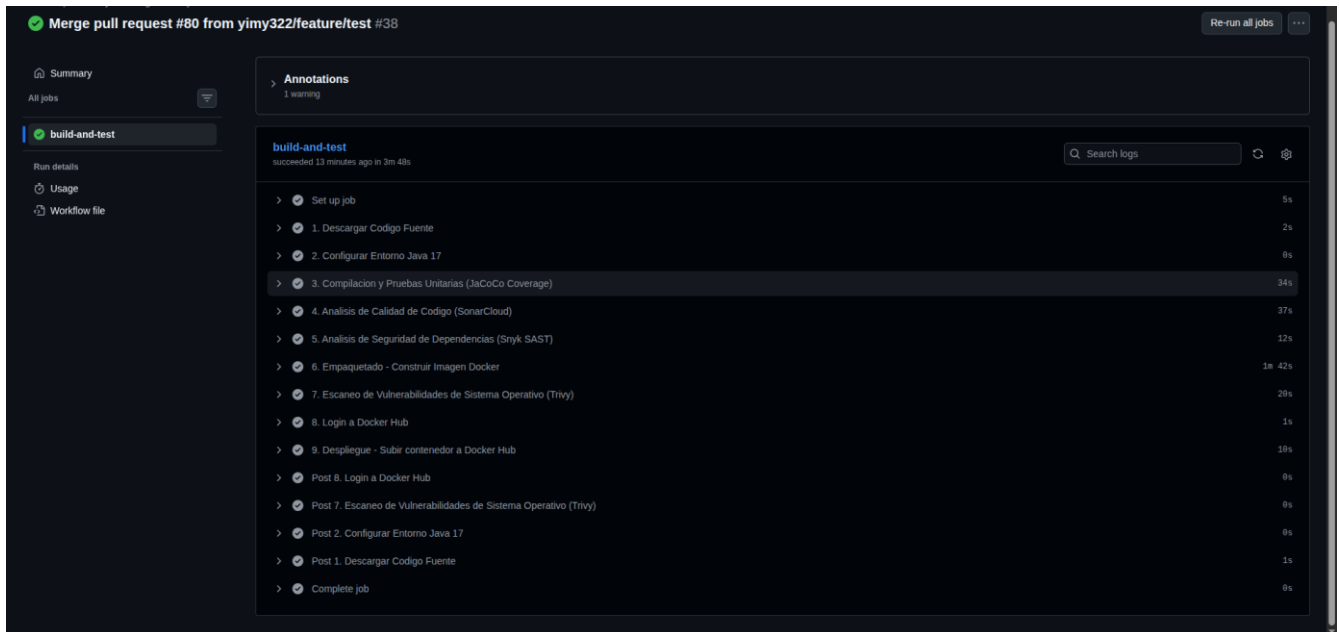
4.13.11. Ejecución completa del pipeline (GitHub Actions)

La siguiente imagen muestra la ejecución exitosa y completa del pipeline, disparada por el merge del pull request #80 (feature/test) hacia la rama principal. El job build-and-test finalizó correctamente en un tiempo total de 3 minutos 48 segundos, ejecutando en orden los nueve pasos definidos en el workflow:

1. Descargar código fuente — 2s
2. Configurar entorno Java 17 — 8s
3. Compilación y pruebas unitarias (JaCoCo Coverage) — 34s
4. Análisis de calidad de código (SonarCloud) — 37s
5. Análisis de seguridad de dependencias (Snyk SAST) — 12s
6. Empaquetado — construcción de la imagen Docker — 1min 42s
7. Escaneo de vulnerabilidades de sistema operativo (Trivy) — 20s
8. Login a Docker Hub — 1s

9. Despliegue — subir contenedor a Docker Hub — 10s

Todos los pasos se completaron con estado exitoso (check verde), confirmando que el código mergeado pasó correctamente por todas las etapas de integración continua y de seguridad antes de ser publicado como imagen en Docker Hub. Se registró únicamente una advertencia (*warning*) no bloqueante durante la ejecución, sin afectar el resultado final del pipeline.



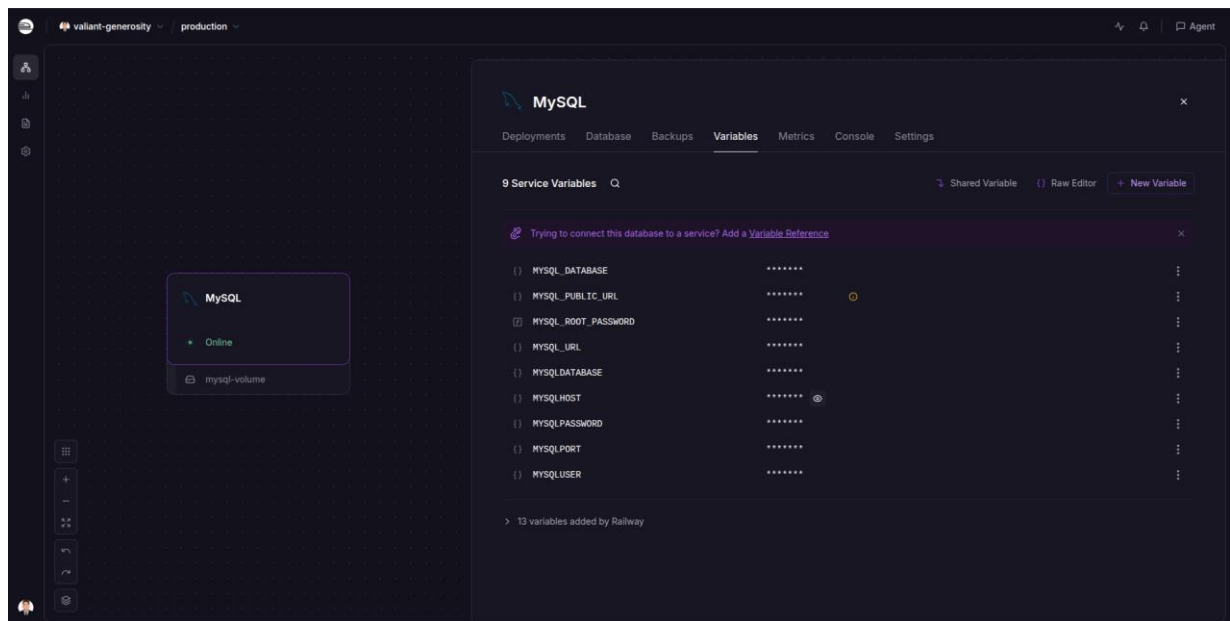
**** Ejecución completa y exitosa del job build-and-test en GitHub Actions, con el detalle de tiempo por cada paso.***

4.13.12. Despliegue en Railway

Como parte del proceso de despliegue continuo, se utilizó Railway como plataforma cloud para publicar la aplicación y conectar el sistema con una base de datos MySQL. A continuación, se presentan las principales etapas realizadas durante el despliegue.

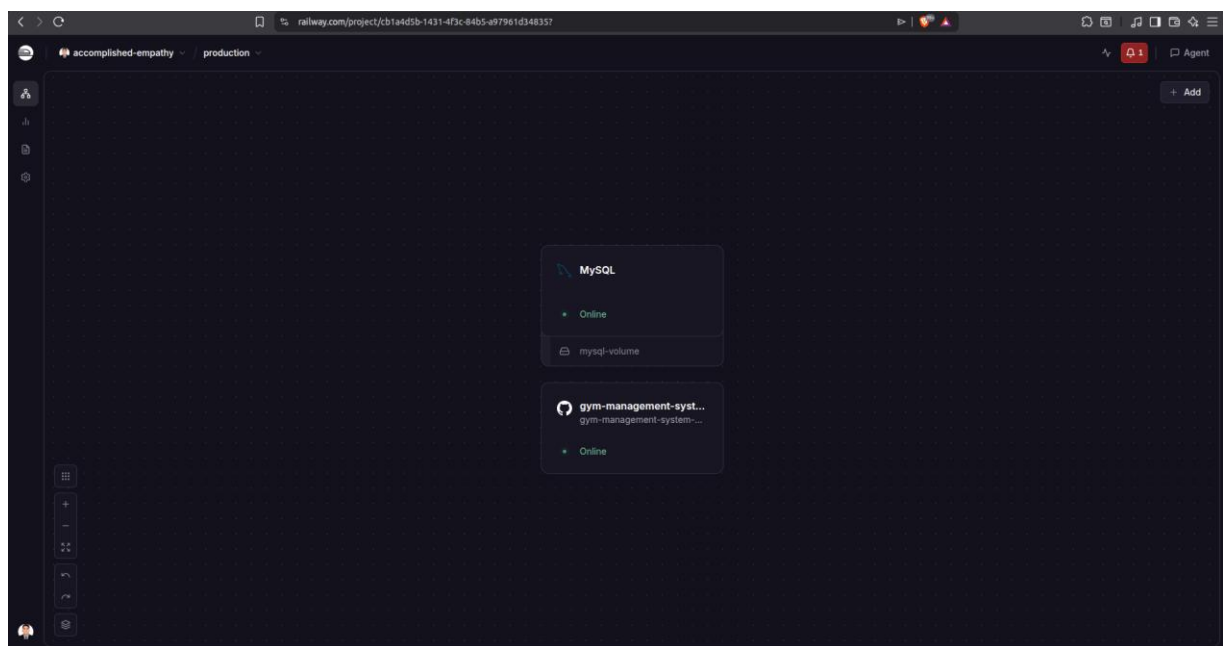
Paso 1: Creación de la Base de Datos

Se creó una instancia de base de datos MySQL dentro de Railway, la cual será utilizada para almacenar la información del sistema.



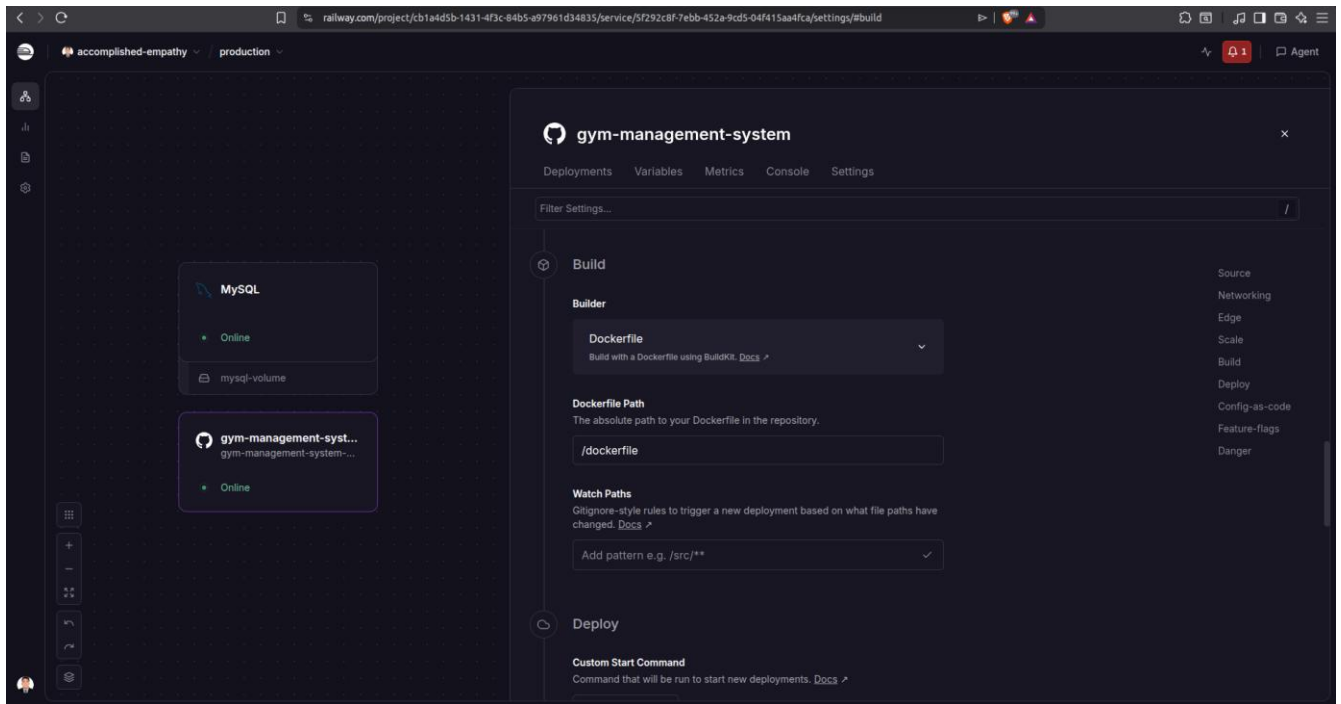
Paso 2: Importación del Repositorio

Posteriormente, se importó el repositorio alojado en GitHub. Railway detectó automáticamente el proyecto y preparó el entorno para su despliegue.



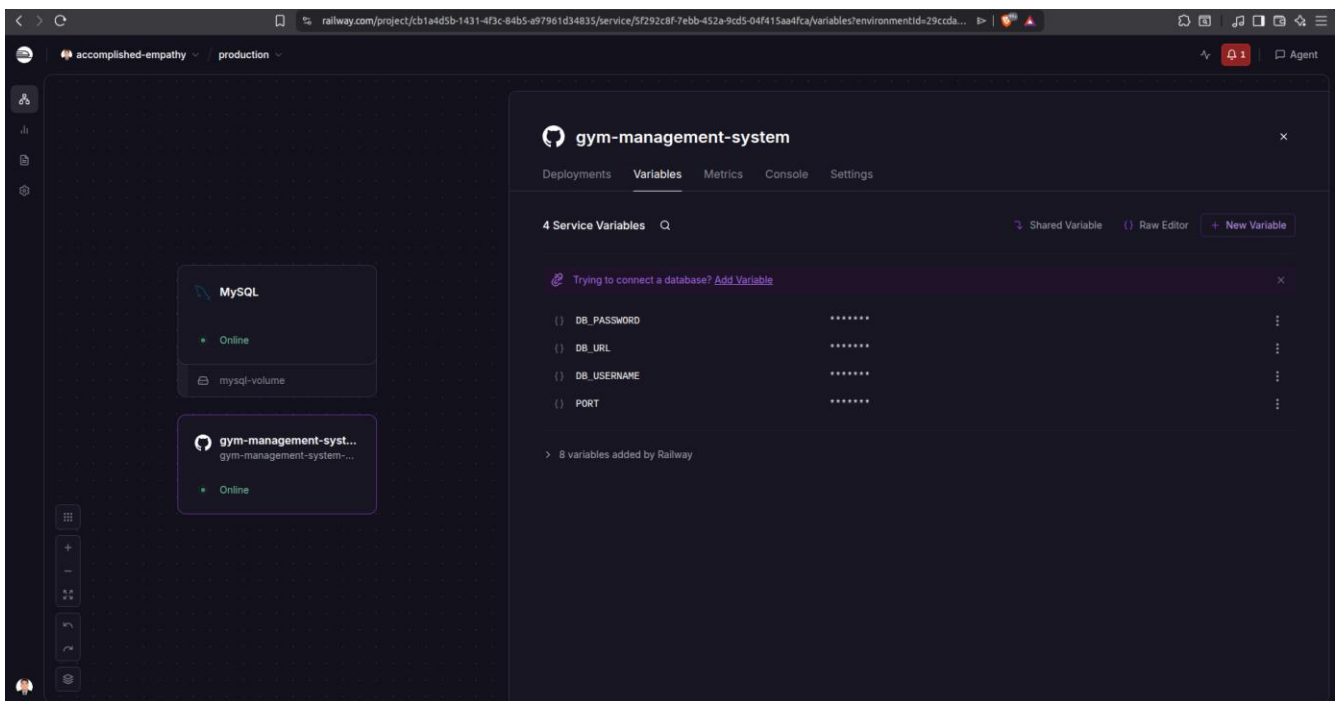
Paso 3: Detección del Dockerfile

Railway identificó automáticamente el Dockerfile configurado en el proyecto, permitiendo construir la aplicación mediante contenedores Docker.



Paso 4: Configuración de Variables de Entorno

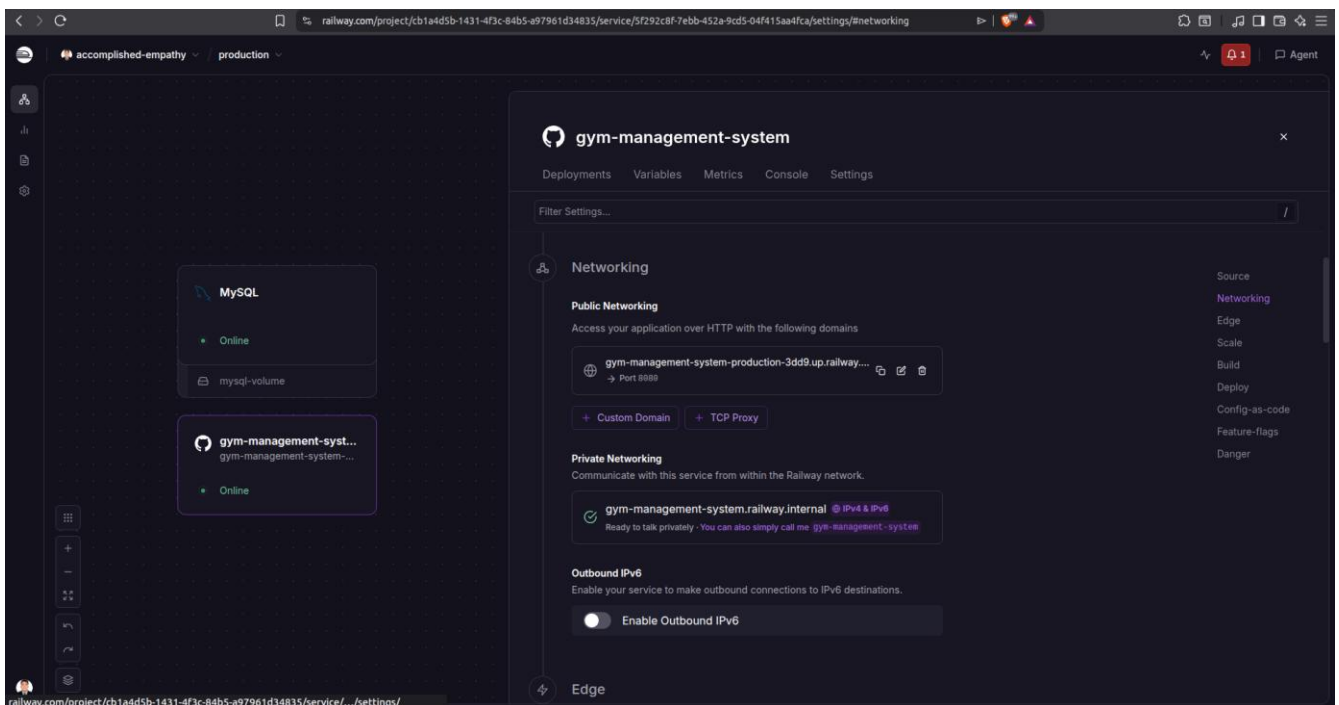
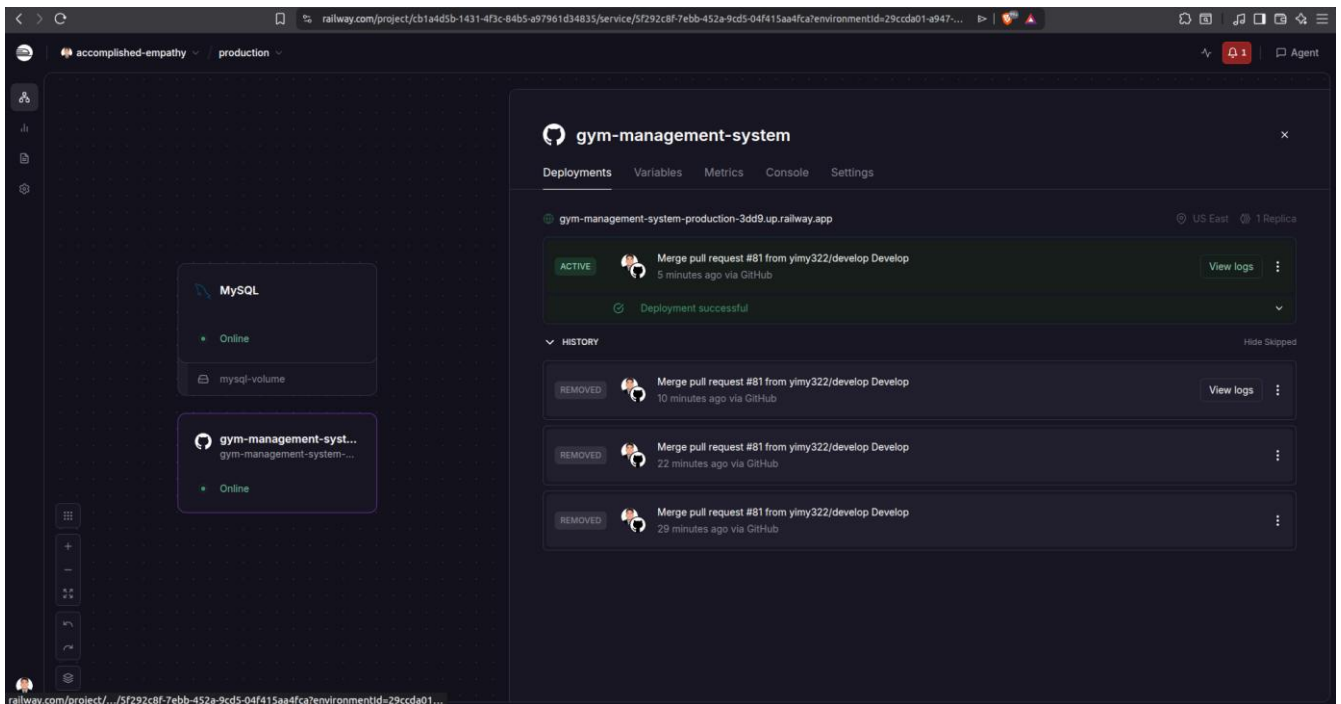
Se configuraron las variables de entorno necesarias para establecer la conexión entre la aplicación y la base de datos MySQL.



Paso 6: Generación del Dominio Público

Tras finalizar el despliegue, Railway generó automáticamente un dominio público para acceder a la

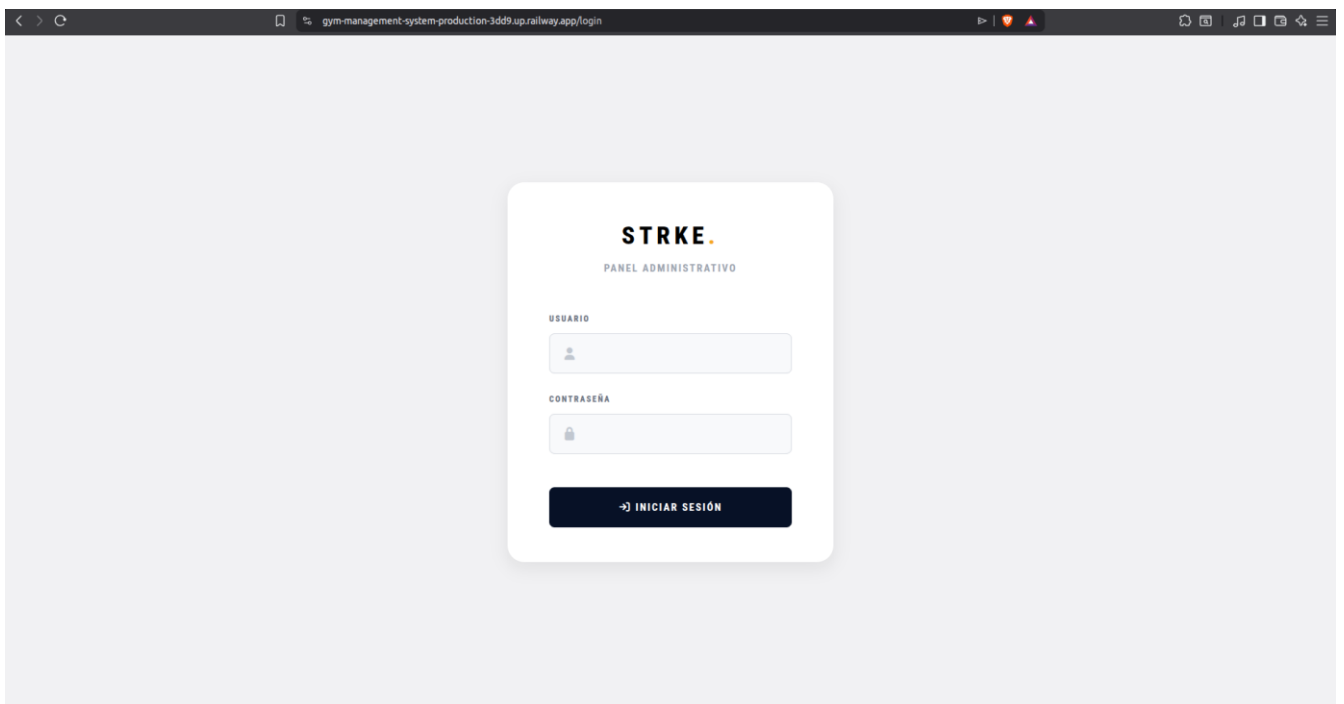
aplicación desde Internet.



Paso 7: Aplicación Desplegada

Finalmente, se verificó el correcto funcionamiento del sistema accediendo a la aplicación mediante la URL pública generada por Railway.

URL del Proyecto en Railway: <https://gym-management-system-production-3dd9.up.railway.app/login>



❖ Credenciales de acceso

Para la validación del despliegue en Railway se utilizó una cuenta de administrador de prueba. Estas credenciales permiten acceder al sistema y verificar el correcto funcionamiento de la aplicación desplegada.



4.14. Herramientas Complementarias

Además de las herramientas principales de control de versiones, gestión y CI/CD, el equipo utilizó un conjunto de herramientas complementarias que apoyaron la comunicación, coordinación, desarrollo y documentación del proyecto.

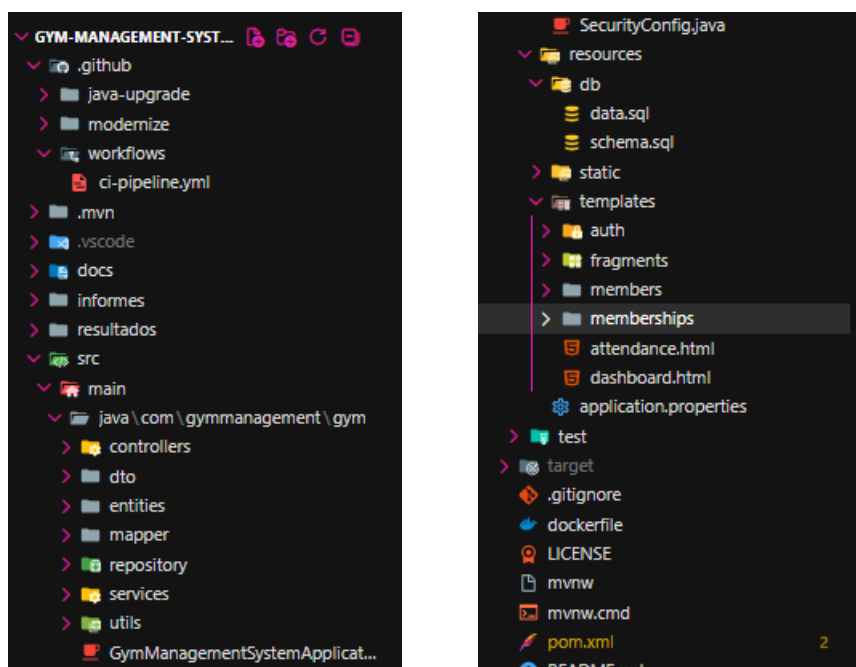
- WhatsApp: utilizado para la comunicación rápida del equipo, coordinación de actividades y seguimiento de avances.
- Zoom: utilizado para reuniones virtuales, revisión de avances y presentación de entregables del proyecto.
- Visual Studio Code: utilizado como entorno de desarrollo integrado (IDE) para la edición del código fuente, así como para la identificación y resolución de conflictos de fusión (merge conflicts) entre ramas.
- Excel: utilizado para organizar la distribución de tareas, asignar responsabilidades y realizar seguimiento del progreso del equipo.
- Word: utilizado para la elaboración del informe final, documentación y recopilación de evidencias del proyecto.
- Canva: utilizado para el diseño y elaboración de las diapositivas para las exposiciones.

CAPÍTULO 5

Implementación y Resultados

5.1. Arquitectura Funcional del sistema

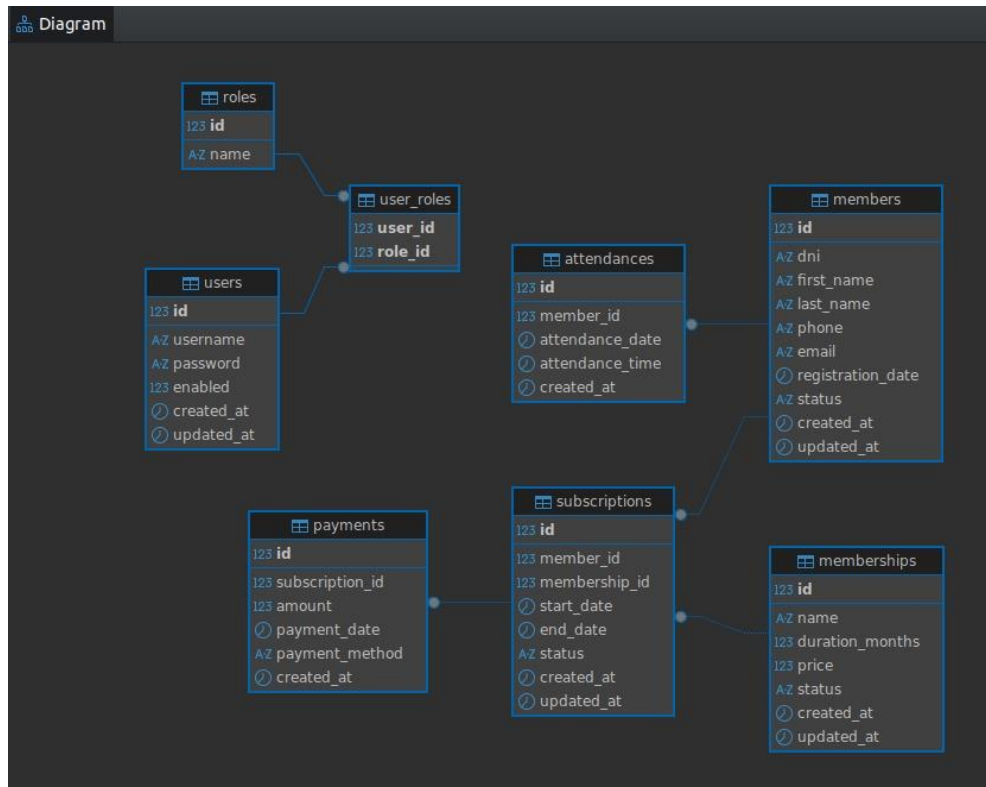
El sistema web fue desarrollado utilizando una arquitectura por capas basada en Spring Boot, permitiendo una adecuada separación de responsabilidades entre los distintos componentes de la aplicación. La estructura implementada organiza la lógica del sistema mediante controladores, servicios, repositorios y entidades, facilitando el mantenimiento, escalabilidad y reutilización del código. Asimismo, incorpora componentes complementarios para la gestión de datos, seguridad, vistas y recursos estáticos, contribuyendo a una mejor organización del proyecto y al desarrollo de una aplicación más robusta y modular.



** Estructura del proyecto*

5.2. Base de Datos

La base de datos fue diseñada bajo un modelo relacional para gestionar la información del gimnasio STRKE. Está compuesta por tablas que permiten administrar afiliados, membresías, suscripciones, pagos y asistencias. Además, incorpora tablas de usuarios y roles para el control de acceso al sistema. Este diseño permite mantener la información organizada, garantizar la integridad de los datos y facilitar las operaciones administrativas del gimnasio.



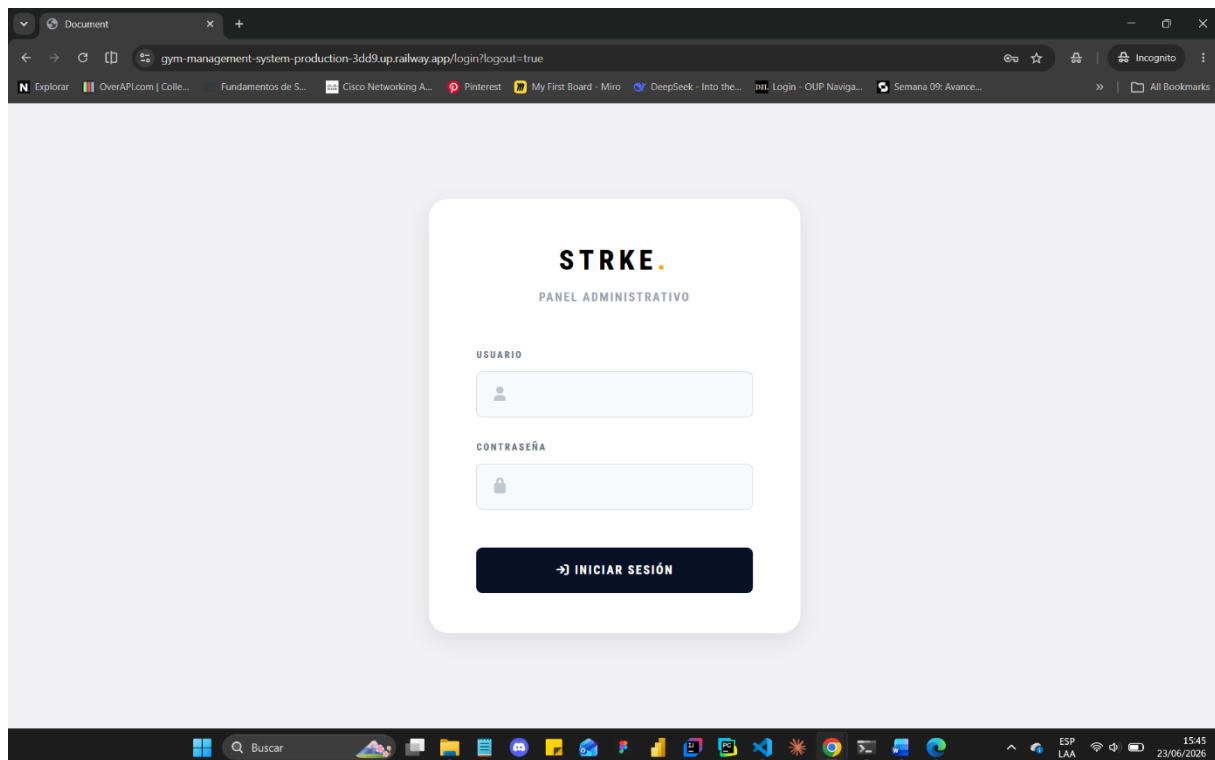
** Diagrama de la base de datos*

5.3. Interfaces implementadas

El sistema web desarrollado para el gimnasio STRKE cuenta con diversas interfaces orientadas a facilitar la gestión administrativa de afiliados, membresías y asistencia. A continuación, se presentan las principales pantallas implementadas durante el desarrollo del proyecto.

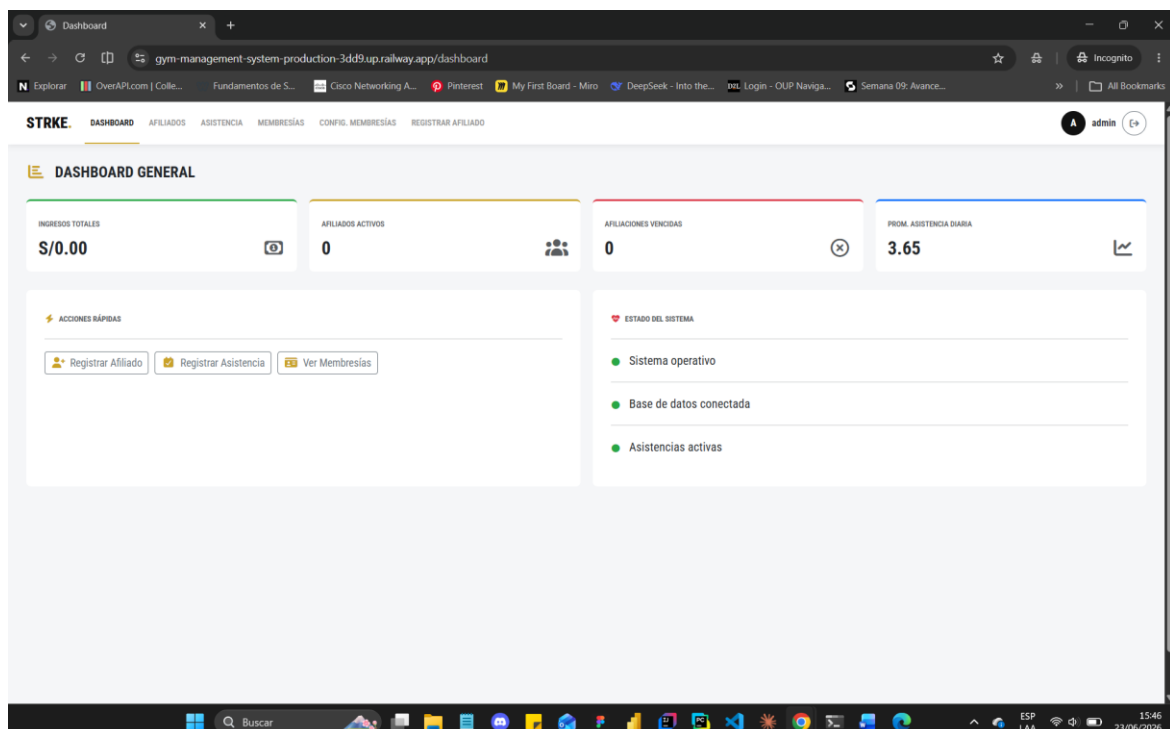
5.3.1. Inicio de Sesión

La interfaz de autenticación permite el acceso seguro de los usuarios autorizados al sistema mediante credenciales de usuario y contraseña.



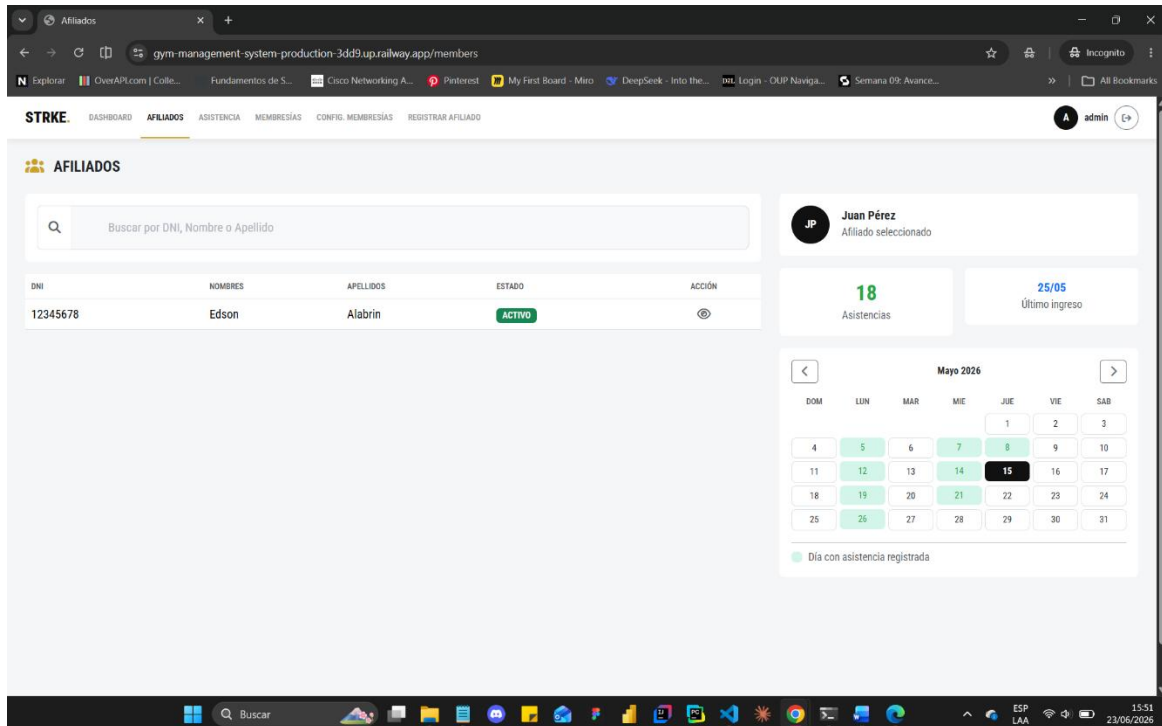
5.3.2. Dashboard General

El dashboard proporciona una visión general de la información más relevante del sistema, mostrando indicadores como ingresos, afiliados activos, membresías vencidas y estadísticas de asistencia.



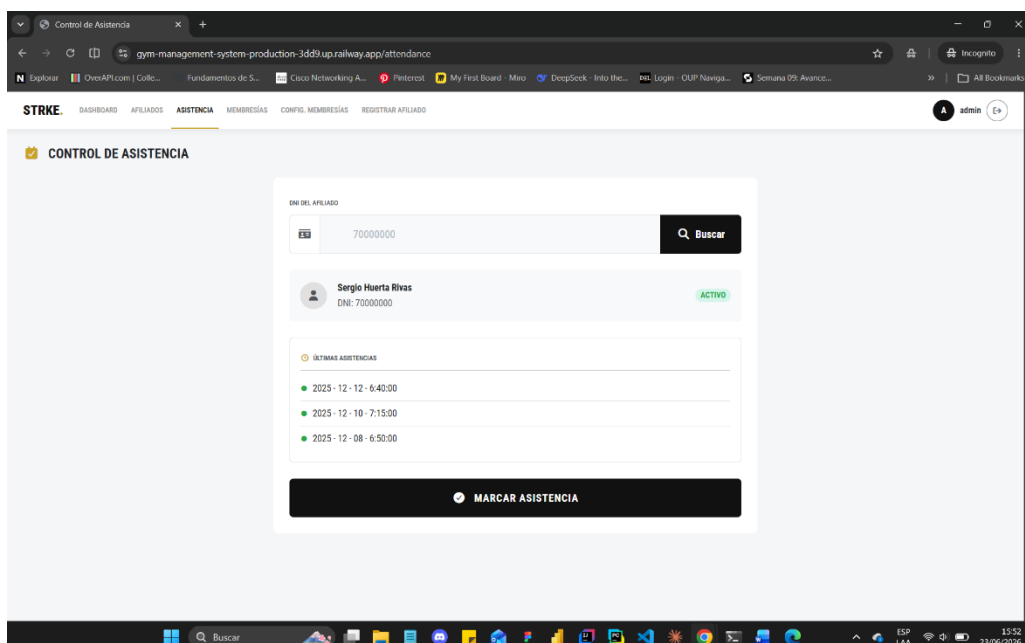
5.3.3. Gestión de Afiliados

Esta interfaz permite consultar la información de los afiliados registrados, visualizar su historial de asistencia y realizar búsquedas mediante DNI o nombre.



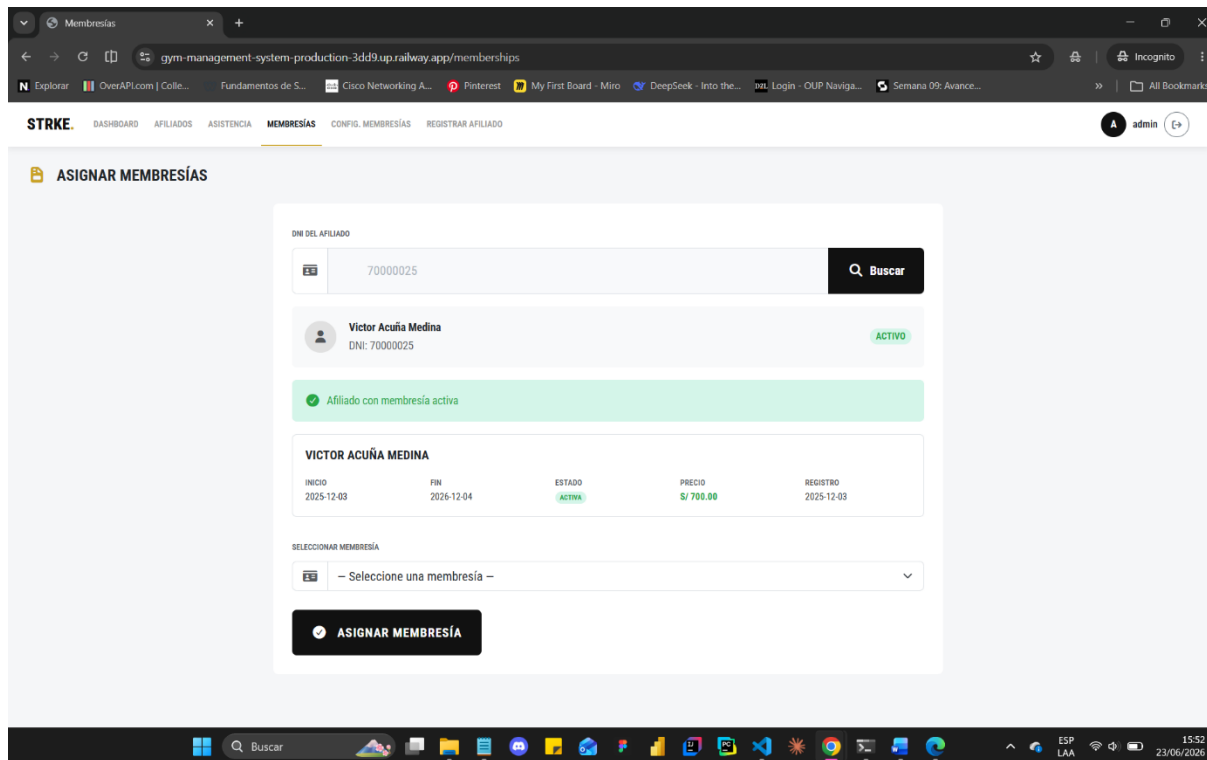
5.3.4. Control de Asistencia

El módulo de asistencia permite registrar y consultar los ingresos de los afiliados al gimnasio, facilitando el seguimiento de la actividad diaria.



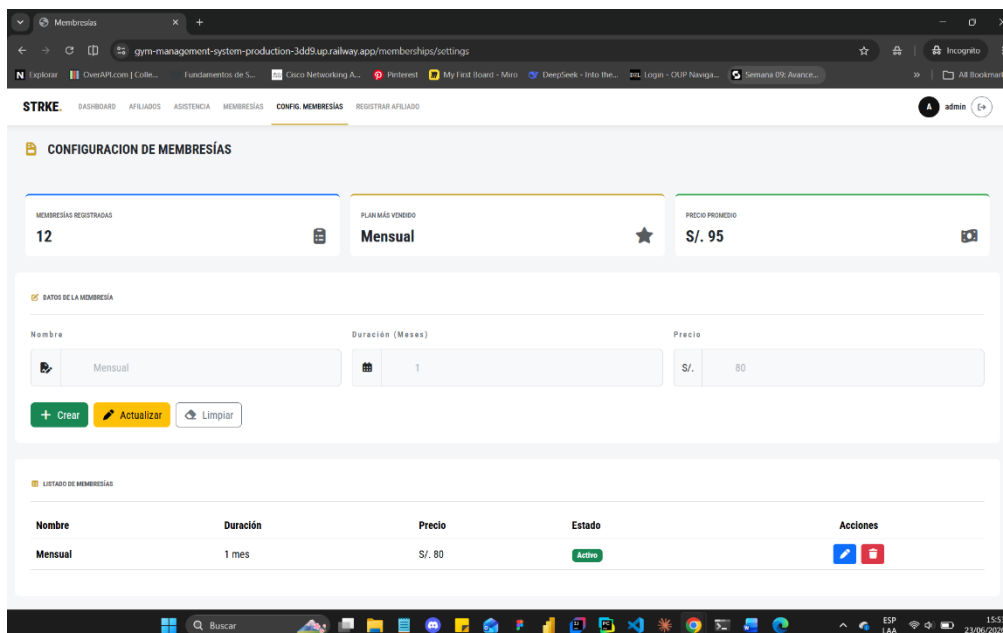
5.3.5. Asignación de Membresías

Esta pantalla permite asociar membresías a los afiliados registrados, verificando el estado actual de cada suscripción y gestionando su vigencia.



5.3.6. Configuración de Membresías

El módulo permite administrar los tipos de membresías disponibles, incluyendo operaciones de registro, actualización y visualización de planes.



5.3.7. Registro de Afiliados

La interfaz de registro permite incorporar nuevos afiliados al sistema mediante el ingreso de información personal básica, facilitando su posterior gestión administrativa.

The screenshot displays a web application interface for managing members. The browser address bar shows the URL: `gym-management-system-production-3dd9.up.railway.app/members/register`. The application header includes the logo 'STRKE' and navigation links: DASHBOARD, AFILIADOS, ASISTENCIA, MEMBRESÍAS, CONFIG. MEMBRESÍAS, and REGISTRAR AFILIADO. The user is logged in as 'admin'.

The main section is titled 'REGISTRO DE AFILIADOS' and contains a 'NUEVO AFILIADO' form. The form fields are as follows:

NUEVO AFILIADO		
DNI	NOMBRES	APELLIDOS
7777777	Nombres	Apellidos
TELEFONO	EMAIL	
999999999	email@ejemplo.com	
<button>Crear afiliado</button>		

Below the form is a 'Lista de afiliados' table:

DNI	NOMBRE	APELLIDOS	ESTADO	ACCIONES
12345678	Edson	Alabrin	ACTIVO	

Conclusiones

- Se logró desarrollar un sistema web de gestión para el gimnasio STRKE aplicando herramientas de desarrollo colaborativo que permitieron organizar, controlar y dar seguimiento a cada etapa del proyecto de manera eficiente.
- La utilización de Git, GitHub y el modelo Git Flow facilitó la gestión de versiones, el trabajo en equipo y la trazabilidad de los cambios realizados, permitiendo mantener un historial organizado del desarrollo del software.
- La implementación de Pull Requests, revisiones de código, GitHub Projects, Jira y Trello contribuyó a mejorar la coordinación entre los integrantes del equipo, así como el seguimiento y control de las actividades asignadas.
- La integración de un pipeline de Integración Continua y DevSecOps mediante GitHub Actions, junto con herramientas como JUnit, Mockito, JaCoCo, SonarCloud, Snyk y Trivy, permitió automatizar procesos de validación, calidad y seguridad, reduciendo errores y fortaleciendo la confiabilidad de la aplicación.
- El uso de Docker para la contenerización y Railway como plataforma de despliegue en la nube permitió preparar el sistema para entornos reales de ejecución, demostrando la importancia de la automatización y las tecnologías cloud dentro del ciclo de vida moderno del desarrollo de software.

Referencias Bibliográficas

- China, C. R., & Goodwin, M. (2025, noviembre 28). ¿Qué es la integración continua? IBM. <https://www.ibm.com/es-es/think/topics/continuous-integration>
- Forage. (2024, febrero 27). What is Git? Definition and how to use it. Forage. <https://www.theforage.com/blog/skills/what-is-git>
- ¿Qué es Bootstrap y cómo funciona este framework? (s.f.). Santander Open Academy. Recuperado el 18 de junio de 2026, de <https://www.santanderopenacademy.com/es/blog/que-es-bootstrap.html>
- ¿Qué es Java y por qué lo necesito? (s.f.). Java.com. Recuperado el 18 de junio de 2026, de https://www.java.com/es/download/help/whatis_java.html
- ¿Qué es Java Spring Boot? (2025, noviembre 26). IBM. <https://www.ibm.com/mx-es/think/topics/java-spring-boot>
- ¿Qué es un sistema web? (s.f.). Creasystem.net. Recuperado el 18 de junio de 2026, de <https://www.creasystem.net/posts/que-es-un-sistema-web>
- Qué es DevSecOps. (s.f.). Fortinet. Recuperado el 18 de junio de 2026, de <https://www.fortinet.com/lat/resources/cyberglossary/devsecops>
- Salgado, L. G. (2022, abril 11). ¿Qué es MySQL Workbench? KeepCoding Bootcamps. <https://keepcoding.io/blog/que-es-mysql-workbench/>
- Salgado, L. G. (2024, octubre 22). Thymeleaf de Spring Boot: integra el MVC en tu diseño web. KeepCoding Bootcamps. <https://keepcoding.io/blog/que-es-el-thymeleaf-de-spring-boot/>
- TheBridge. (s.f.). ¿Qué es GitHub? Recuperado el 18 de junio de 2026, de <https://thebridge.tech/blog/que-es-git-hub/>
- What is Git Flow. (2021, marzo 24). GitKraken. <https://www.gitkraken.com/learn/git/git-flow>